

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HCM
KHOA ĐIỆN - ĐIỆN TỬ



HCMUTE

BÁO CÁO ĐỒ ÁN I

**ĐỀ TÀI: ỨNG DỤNG MÔ HÌNH YOLOv7-MobileNetV3
TRÊN THIẾT BỊ BIÊN ĐỀ NHẬN ĐIỆN ĐÁM CHÁY**

GIÁO VIÊN HƯỚNG DẪN: PGS.TS Võ Minh Huân
SINH VIÊN THỰC HIỆN: Đoàn Minh Duy Bình
MÃ SỐ SINH VIÊN: 23139005

TP. Hồ Chí Minh, tháng 7 năm 2026

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HCM
KHOA ĐIỆN - ĐIỆN TỬ**



BÁO CÁO ĐỒ ÁN I

**ĐỀ TÀI: ỨNG DỤNG MÔ HÌNH YOLOv7-MobileNetV3
TRÊN THIẾT BỊ BIÊN ĐỀ NHẬN ĐIỆN ĐÁM CHÁY**

**GIÁO VIÊN HƯỚNG DẪN: PGS.TS Võ Minh Huân
SINH VIÊN THỰC HIỆN: Đoàn Minh Duy Bình
MÃ SỐ SINH VIÊN: 23139005**

TP. Hồ Chí Minh, tháng 7 năm 2026

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI.....	7
1.1. Đặt vấn đề và lý do chọn đề tài	7
1.2. Mục tiêu nghiên cứu	7
1.2.1. Mục tiêu tổng quát	7
1.2.2. Mục tiêu cụ thể	8
1.3. Đối tượng và phạm vi nghiên cứu	8
1.3.1. Đối tượng nghiên cứu	8
1.3.2. Phạm vi nghiên cứu	8
1.4. Bố cục của đồ án	9
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG	10
2.1. Tổng quan về Thị giác máy tính và Mạng nén chập (CNN)	10
2.2. Kiến trúc mô hình phát hiện vật thể YOLOv7	11
2.3. Khối tích chập nhẹ MobileNetV3 (Backbone)	12
2.3.1. Depthwise Separable Convolutions	15
2.3.2. Sự kết hợp lai (Hybrid) YOLOv7-MobileNetV3	16
2.4. Kỹ thuật tối ưu hóa Trí tuệ nhân tạo biên	17
2.4.1. Lượng tử hóa mô hình	17
2.4.2. Định dạng chuẩn mở ONNX	17
2.4.3. Khung suy luận NCNN tối ưu cho vi kiến trúc ARM	18
2.5. Tổng quan phần cứng Raspberry Pi 4 và Jetson Nano	18
CHƯƠNG 3: THU THẬP DỮ LIỆU VÀ HUẤN LUYỆN MÔ HÌNH	20
3.1. Chuẩn bị tập dữ liệu (Dataset)	20
3.1.1. Nguồn dữ liệu và phân loại	20
3.1.2. Kỹ thuật tiền xử lý và căn chỉnh dữ liệu	21
3.2. Xây dựng môi trường huấn luyện	22
3.2.1. Thiết lập máy chủ Kaggle (GPU)	22
3.2.2. Cấu hình file YAML định tuyến dữ liệu	22
3.3. Quá trình huấn luyện mô hình	23
3.3.1. Tùy chỉnh siêu tham số	23
3.3.2. Biểu đồ suy hao trong quá trình huấn luyện (Loss function curves)	24
3.4. Xuất và đóng gói mô hình	25
3.4.1. Chuyển đổi sang định dạng ONNX (FP16)	25
3.4.2. Biên dịch sang định dạng NCNN cho chip ARM	25
CHƯƠNG 4: ĐÁNH GIÁ VÀ SO SÁNH HIỆU NĂNG MÔ HÌNH	26
4.1. Các phương pháp và thang đo đánh giá	26
4.2. Kiểm thử và so sánh độ chính xác	26
4.2.1. Ma trận nhầm lẫn	28
4.2.2. Bảng kết quả so sánh mAP giữa mô hình gốc (.pt) và ONNX ...	29
4.3. Kiểm thử và so sánh tốc độ suy luận (Inference Speed)	30
4.3.1. Kết quả đo lường thời gian xử lý (ms/ảnh)	30
4.3.2. Đánh giá tác động của việc lượng tử hóa đến tốc độ	30
CHƯƠNG 5: TRIỂN KHAI THỰC TẾ TRÊN HỆ THỐNG NHÚNG	32
6.1. Thiết lập phần cứng và môi trường hệ điều hành	32

6.1.1. Vận chuyển dữ liệu qua giao thức SSH	32
6.1.2. Cài đặt thư viện suy luận tối ưu	33
6.1.3. Cài đặt thư viện suy luận tối ưu (CPU-only)	33
6.2. Xây dựng kịch bản suy luận (Inference Script)	34
6.2.1. Cấu trúc mã nguồn nhận diện tự động	34
6.2.2. Xử lý đầu vào/đầu ra và vẽ Bounding Box	34
6.3. Đánh giá kết quả chạy thực tế trên Raspberry Pi 4	35
6.3.1. Phân tích tốc độ khung hình (FPS) thu được	35
6.3.2. Hình ảnh kết quả nhận diện thực tế	35
CHƯƠNG 7: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	39
7.1. Tổng kết những kết quả đạt được	39
7.2. Hạn chế của đề tài	39
7.3. Hướng phát triển trong tương lai	39
TÀI LIỆU THAM KHẢO	41

BÁO CÁO CUỐI KỲ ĐỒ ÁN 1
HỌC KỲ 2 NĂM HỌC 2025–2026

1. **Mã lớp môn học:** : SEPR319164
2. **Giảng viên hướng dẫn:** Pgs.Ts. Võ Minh Huân
3. **Tên đề tài:** Ứng dụng mô hình YOLOv7-MobileNetV3 trên thiết bị biên để nhận diện đám cháy

Nhận xét của giảng viên

.....

.....

.....

.....

.....

.....

.....

.....

.....

Ngày tháng năm 2026

Giáo viên chấm điểm

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin gửi lời cảm ơn chân thành đến Pgs.Ts. Võ Minh Huân giảng viên phụ trách Đồ án 1. Trong suốt quá trình học tập, Thầy đã rất tâm huyết dạy và hướng dẫn cho chúng tôi nhiều điều bổ ích trong môn học và kỹ năng làm một bài nghiên cứu để chúng tôi có đủ kiến thức thực hiện bài báo cáo này.

Tuy nhiên vì kiến thức bản thân còn nhiều hạn chế và sự tìm hiểu chưa sâu sắc nên không tránh khỏi những thiếu sót. Mong Thầy sẽ châm chước và cho chúng em những lời góp ý để bài nghiên cứu của chúng tôi sẽ hoàn thiện hơn. Một lần nữa, chúng tôi xin gửi lời cảm ơn sâu sắc đến Thầy và chúc Thầy luôn mạnh khỏe, hạnh phúc và thành công trong sự nghiệp.

TP. Hồ Chí Minh, tháng 7 năm 2026

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1. Đặt vấn đề và lý do chọn đề tài

Trong những năm gần đây, hỏa hoạn và cháy rừng đang trở thành một trong những thảm họa gây thiệt hại nghiêm trọng nhất về người, tài sản và hệ sinh thái. Các hệ thống cảnh báo cháy truyền thống (như cảm biến khói, cảm biến nhiệt tĩnh) hoặc camera giám sát cố định thường bộc lộ nhiều hạn chế: phạm vi quan sát hẹp, độ trễ cảnh báo cao và gần như vô hiệu tại các khu vực địa hình hiểm trở, hẻo lánh.

Đề giải quyết bài toán điểm mù không gian, Thiết bị bay không người lái kết hợp với công nghệ Thị giác máy tính đang nổi lên như một giải pháp đột phá. Drone cung cấp góc nhìn toàn cảnh từ trên cao, tính cơ động vượt trội, cho phép tiếp cận và quét các khu vực rộng lớn một cách nhanh chóng. Tuy nhiên, việc ứng dụng Trí tuệ nhân tạo lên Drone lại vấp phải một rào cản kỹ thuật lớn: các mô hình học sâu hiện đại (như họ YOLO nguyên bản) đòi hỏi năng lực tính toán khổng lồ từ các bộ xử lý đồ họa mạnh mẽ. Việc truyền dữ liệu video trực tiếp từ Drone về máy chủ đám mây để xử lý sinh ra độ trễ lớn, tiêu tốn băng thông và đặc biệt dễ tê liệt hoàn toàn khi hạ tầng mạng viễn thông bị phá hủy trong thảm họa.

Nhằm giải quyết triệt để nút thắt này, tư duy đưa năng lực xử lý trí tuệ nhân tạo xuống trực tiếp thiết bị biên trở thành xu hướng tất yếu của các hệ thống IoT hiện đại. Bằng cách xử lý dữ liệu ngay tại chỗ, hệ thống đảm bảo tính thời gian thực, hoạt động độc lập không cần Internet và tối ưu năng lượng. Mặc dù vậy, thiết bị một số biên (như Raspberry Pi) lại có tài nguyên bộ nhớ và năng lực CPU hạn chế.

Từ những thách thức thực tiễn đó, đề tài “Ứng dụng mô hình YOLOv7-MobileNetV3 trên thiết bị biên để nhận diện đám cháy” được lựa chọn. Đề tài tập trung nghiên cứu việc lai tạo kiến trúc mô hình (sử dụng MobileNetV3 làm bộ khung trích xuất đặc trưng gọn nhẹ) kết hợp với các kỹ thuật lượng tử hóa. Qua đó, đề tài hướng tới việc chứng minh khả năng triển khai các mô hình học sâu mạnh mẽ lên các thiết bị nhúng giá rẻ, giải quyết bài toán cân bằng giữa độ chính xác nhận diện và giới hạn phần cứng, góp phần nâng cao hiệu quả cảnh báo cháy và tìm kiếm cứu nạn chủ động.

1.2. Mục tiêu nghiên cứu

1.2.1. Mục tiêu tổng quát

Nghiên cứu, huấn luyện và tối ưu hóa mô hình nhận diện vật thể YOLOv7 kết hợp bộ khung MobileNetV3 nhằm phát hiện sớm khói, đám cháy và con người từ hình ảnh trên cao. Đồng thời, triển khai thành công mô hình đã

tối ưu lên vi tính nhúng (Raspberry Pi 4) để thực hiện quá trình suy luận độc lập tại biên với tốc độ và độ chính xác đáp ứng yêu cầu thực tế.

1.2.2. Mục tiêu cụ thể

Để đạt được mục tiêu tổng quát, đề tài đặt ra các mục tiêu cụ thể cần giải quyết như sau:

- Thu thập và tiền xử lý dữ liệu bằng cách xây dựng tập dữ liệu hình ảnh chứa các đối tượng khói, lửa và người được chụp/quay từ góc nhìn của Drone. Thực hiện gán nhãn và tăng cường dữ liệu để tăng tính khái quát cho mô hình.
- Huấn luyện mô hình, thiết lập môi trường và huấn luyện mô hình YOLOv7-MobileNetV3. Đánh giá độ chính xác của mô hình thông qua các chỉ số chuyên chuẩn như Precision, Recall, mAP@50 và mAP@50-95.
- Chuyển đổi trọng số mô hình gốc sang các định dạng tối ưu cho thiết bị biên (ONNX, NCNN), áp dụng kỹ thuật lượng tử hóa (FP16) nhằm giảm dung lượng và giảm tải khối lượng tính toán mà không làm suy giảm đáng kể độ chính xác.
- Triển khai trên hệ thống nhúng như xây dựng kịch bản vận hành, cấu hình môi trường phần mềm tối ưu trên Raspberry Pi 4 để loại bỏ các tài nguyên dư thừa, tập trung vào khả năng xử lý bằng CPU.
- Đánh giá hiệu năng thực tế, đo lường, so sánh hiệu suất giữa mô hình gốc chạy trên PC và mô hình tối ưu chạy trên mạch nhúng dựa trên tốc độ khung hình (FPS) và thời gian phản hồi (ms), từ đó khẳng định tính khả thi của giải pháp Edge AI.

1.3. Đối tượng và phạm vi nghiên cứu

1.3.1. Đối tượng nghiên cứu

Để giải quyết bài toán nhận diện đám cháy tại biên, đề án tập trung nghiên cứu bốn đối tượng chính. Đầu tiên là kiến trúc mạng nén chập bao gồm mô hình học sâu YOLOv7 và các khối tích chập tách biệt chiều sâu của MobileNetV3. Thứ hai là các kỹ thuật tối ưu hóa thông qua quy trình chuyển đổi và nén trọng số mô hình từ định dạng PyTorch sang các định dạng chuẩn mở ONNX và khung suy luận chuyên dụng NCNN cho vi kiến trúc ARM. Thứ ba là hệ thống phần cứng nhúng, cụ thể là vi máy tính Raspberry Pi 4 đóng vai trò là thiết bị biên thực hiện quá trình suy luận độc lập. Cuối cùng là tập dữ liệu huấn luyện bao gồm các hình ảnh và video chứa đối tượng khói, lửa và con người được ghi lại từ góc nhìn từ trên cao của thiết bị bay không người lái.

1.3.2. Phạm vi nghiên cứu

Do giới hạn về thời gian thực hiện và nguồn lực phần cứng, đề tài được giới hạn trong các phạm vi cụ thể. Xét về mặt dữ liệu, đồ án chỉ tập trung nhận diện ba nhóm đối tượng đặc thù phục vụ công tác cứu hộ và cảnh báo cháy bao gồm khói, lửa và người. Các hình ảnh này được thu thập chủ yếu mô phỏng góc nhìn thẳng từ trên cao của thiết bị bay không người lái. Xét về mặt phần cứng, toàn bộ quá trình huấn luyện được thực hiện trên nền tảng đám mây thông qua máy chủ đồ họa Kaggle. Tuy nhiên, quá trình triển khai và đánh giá thực tế chỉ được thực hiện trên vi xử lý trung tâm của Raspberry Pi 4 mà không sử dụng thêm các bộ gia tốc trí tuệ nhân tạo ngoại vi. Xét về giới hạn tổng thể, đồ án đi sâu vào luồng xử lý nhận diện hình ảnh, tối ưu hóa thuật toán tại thiết bị biên và đánh giá hiệu năng hệ thống nhúng, hoàn toàn không bao gồm việc chế tạo cơ khí, lập trình hệ thống điều khiển bay hay lắp đặt vật lý lên một thiết bị bay thực tế.

1.4. Bố cục của đồ án

Nội dung của đồ án được trình bày logic, liên mạch từ cơ sở lý thuyết đến thực nghiệm, bao gồm sáu chương chính.

Chương 1 trình bày tổng quan đề tài, nêu rõ lý do lựa chọn, tính cấp thiết của việc kết hợp thiết bị bay không người lái và trí tuệ nhân tạo biên trong cảnh báo cháy, đồng thời xác định mục tiêu, đối tượng và phạm vi giới hạn của nghiên cứu.

Chương 2 cung cấp nền tảng kiến thức về mạng nén chập, đi sâu phân tích kiến trúc YOLOv7, sự ưu việt của khối MobileNetV3 trong việc giảm thiểu tham số, cũng như các kỹ thuật lượng tử hóa mô hình phục vụ hệ thống nhúng.

Chương 3 trình bày chi tiết quy trình chuẩn bị tập dữ liệu, thiết lập môi trường huấn luyện trên đám mây và các bước chuyển đổi, đóng gói mô hình thành các định dạng siêu nhẹ.

Chương 4 tập trung vào việc thực hiện kiểm thử độc lập trên máy tính cá nhân, từ đó phân tích và so sánh các chỉ số độ chính xác cùng tốc độ xử lý giữa mô hình nguyên bản và mô hình đã được lượng tử hóa để khẳng định tính hiệu quả của thuật toán nén.

Chương 5 mô tả các bước thiết lập môi trường hệ điều hành trên Raspberry Pi 4, cách khắc phục các giới hạn tài nguyên bộ nhớ và xây dựng kịch bản chạy độc lập, qua đó đánh giá tốc độ khung hình và độ trễ phản hồi trong điều kiện thực tế.

Cuối cùng, Chương 6 tổng kết lại các kết quả đã đạt được so với mục tiêu đề ra ban đầu, phân tích những hạn chế còn tồn đọng và đề xuất các phương án mở rộng, phát triển đồ án trong tương lai.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1. Tổng quan về Thị giác máy tính và Mạng nén chập (CNN)

Thị giác máy tính là một lĩnh vực chuyên sâu thuộc trí tuệ nhân tạo, tập trung vào việc mô phỏng khả năng nhìn và nhận thức của con người thông qua các hệ thống máy tính. Mục tiêu cốt lõi của lĩnh vực này là phân tích, hiểu và trích xuất các thông tin có ý nghĩa từ hình ảnh kỹ thuật số, video hoặc các luồng dữ liệu thị giác khác. Từ những dữ liệu đã được xử lý này, máy móc có thể tự động đưa ra các quyết định hoặc hành động tương ứng mà không cần sự can thiệp liên tục của con người. Trong các hệ thống giám sát và cảnh báo tự động, thị giác máy tính đóng vai trò như một hệ thống giác quan điện tử, giúp thiết bị liên tục quan sát hiện trường và chủ động nhận diện các yếu tố bất thường hoặc nguy hiểm.

Để giải quyết các bài toán phức tạp trong thị giác máy tính, mạng nén chập đã ra đời và nhanh chóng trở thành một trong những kiến trúc học sâu mạnh mẽ nhất. Đây là một lớp mạng nơ-ron nhân tạo được thiết kế chuyên biệt nhằm xử lý các loại dữ liệu có cấu trúc dạng lưới, tiêu biểu nhất là hình ảnh hai chiều dưới dạng các ma trận điểm ảnh. Điểm khác biệt mang tính đột phá của kiến trúc này so với các mạng nơ-ron truyền thống nằm ở việc ứng dụng phép toán tích chập. Phép toán này cho phép mạng tự động trích xuất các đặc trưng không gian từ mức độ thấp như đường viền, góc cạnh, vân bề mặt cho đến các đặc trưng mức độ cao như hình dáng toàn vẹn của một vật thể một cách hoàn toàn tự động trong quá trình huấn luyện.

Cấu trúc điển hình của một mạng nén chập thường bao gồm ba loại lớp cơ bản là lớp tích chập, lớp tổng hợp và lớp kết nối đầy đủ. Lớp tích chập chịu trách nhiệm dùng các bộ lọc trượt qua toàn bộ không gian bức ảnh để tạo ra các bản đồ đặc trưng, qua đó giữ lại được mối liên hệ không gian giữa các điểm ảnh lân cận. Tiếp theo, lớp tổng hợp sẽ thực hiện nhiệm vụ chắt lọc thông tin quan trọng nhất và giảm kích thước không gian của các bản đồ đặc trưng này. Quá trình tổng hợp giúp hệ thống giảm thiểu đáng kể số lượng tham số cần tính toán và kiểm soát hiệu quả hiện tượng học vẹt. Cuối cùng, luồng thông tin đã được tinh lọc kỹ lưỡng sẽ được trải phẳng và đưa qua lớp kết nối đầy đủ để tính toán xác suất, giúp mạng đưa ra kết quả dự đoán cuối cùng như phân loại đối tượng và xác định tọa độ không gian của vật thể.

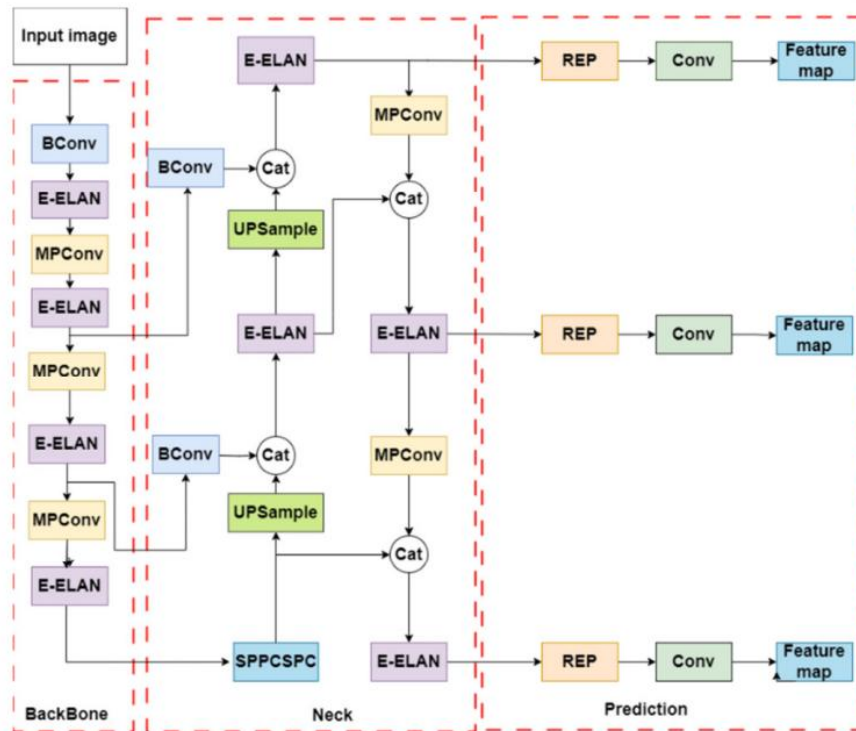
Mặc dù các kiến trúc mạng nén chập truyền thống mang lại độ chính xác rất cao, nhưng chúng thường đi kèm với khối lượng tính toán khổng lồ và yêu cầu bộ nhớ rất lớn. Những đặc tính này tạo ra một rào cản kỹ thuật rất lớn khi muốn triển khai các mô hình học sâu lên các nền tảng thiết bị di động hoặc máy tính nhúng có năng lực xử lý hạn chế. Thực tế này đã thúc đẩy sự ra đời của các kỹ thuật nén mô hình và các kiến trúc mạng học sâu siêu nhẹ, làm tiền đề quan trọng cho việc đưa trí tuệ nhân tạo rời khỏi các máy chủ đồ họa đắt tiền để hoạt động trực tiếp ngay tại các thiết bị biên.

2.2. Kiến trúc mô hình phát hiện vật thể YOLOv7

Mô hình YOLOv7 là một phiên bản cải tiến mạnh mẽ thuộc chuỗi thuật toán phát hiện vật thể YOLO, được thiết kế nhằm đạt được sự cân bằng tối ưu giữa độ chính xác nhận diện và hiệu năng hoạt động. Sự cân bằng này có được thông qua việc tích hợp các phương pháp tiên tiến như tái tham số hóa tích chập, mở rộng quy mô sử dụng các mô hình dựa trên phép nối và mạng chú ý tầm xa hiệu quả mở rộng hay còn gọi là E-ELAN.

Về mặt cấu trúc, mạng YOLOv7 được tổ chức thành bốn thành phần chính bao gồm phần đầu vào, phần xương sống, phần cổ và phần dự đoán. Thành phần đầu vào đảm nhận nhiệm vụ điều chỉnh kích thước của các hình ảnh đầu vào sao cho phù hợp với yêu cầu xử lý của phần xương sống. Tiếp theo, phần xương sống bao gồm các lớp tích chập như E-ELAN và MPConv, trong đó đặc biệt nhấn mạnh vào phân đoạn BConv. Phân đoạn BConv là sự kết hợp chặt chẽ giữa một lớp tích chập với lớp chuẩn hóa theo lô và hàm kích hoạt LeakyReLU, đóng vai trò thiết yếu trong việc trích xuất các đặc trưng hình ảnh ở nhiều tỷ lệ khác nhau một cách vô cùng hiệu quả.

Tại vị trí trung tâm, phần cổ của mô hình sử dụng mạng kim tự tháp đặc trưng kết hợp với tính năng tổng hợp đường dẫn nhằm hòa trộn các đặc trưng được trích xuất từ nhiều độ sâu khác nhau. Quá trình này giúp nâng cao đáng kể khả năng phân tích hình ảnh ở nhiều quy mô của toàn bộ hệ thống. Cuối cùng, thành phần dự đoán chịu trách nhiệm tinh chỉnh số lượng kênh cho các bản đồ đặc trưng, vốn đã được tối ưu hóa thông qua khối REP. Thông qua đó, mạng sử dụng một lớp tích chập chuyên biệt để đưa ra các dự đoán chính xác về điểm số độ tin cậy, phân loại đối tượng và các thông số chi tiết về hộp neo. Nhờ vào khả năng đáp ứng xuất sắc cả hai tiêu chí là xử lý thời gian thực và độ chuẩn xác cao, YOLOv7 được lựa chọn làm nền tảng cốt lõi cho hệ thống nhận diện.

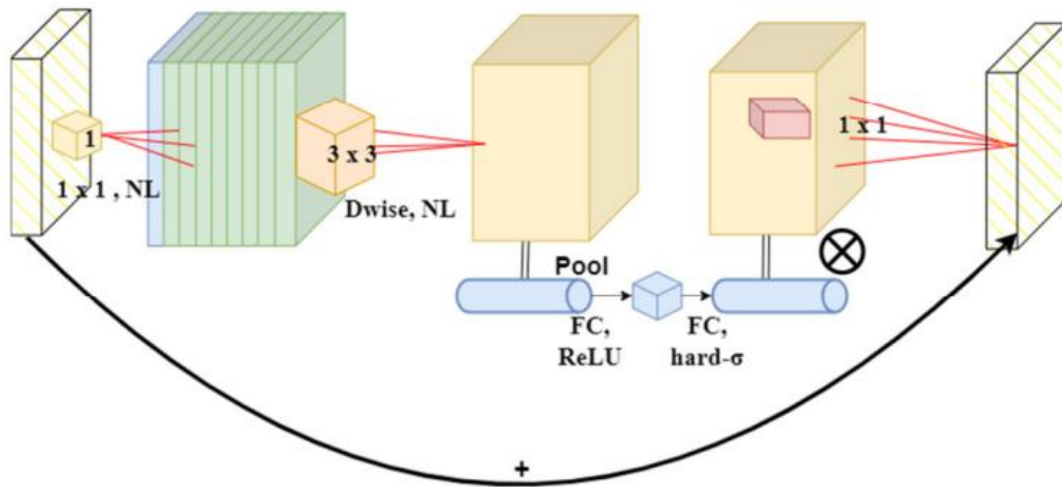


Hình 1: Kiến trúc gốc của mạng YOLOv7

2.3. Khối tích chập nhẹ MobileNetV3 (Backbone)

Kiến trúc MobileNetV3 đánh dấu một bước phát triển then chốt trong quá trình tiến hóa của các mạng học sâu, được tối ưu hóa thông qua quá trình tìm kiếm kiến trúc mạng tự động. Mô hình này kế thừa các tính năng quan trọng từ các phiên bản tiền nhiệm, bao gồm phép tích chập tách biệt chiều sâu và các cấu trúc thắt nút cổ chai tuyến tính kết hợp kết nối phần dư. Một điểm cải tiến đáng chú ý của MobileNetV3 là việc tích hợp các khối nén và kích thích vào bên trong cấu trúc thắt nút cổ chai, giúp nâng cao cả hiệu suất hoạt động lẫn khả năng biểu diễn đặc trưng của mạng. Bên cạnh đó, mô hình thay thế hàm kích hoạt truyền thống bằng hàm h-swish, một giải pháp thay thế hiệu quả hơn nhằm giảm thiểu khối lượng tính toán khổng lồ sinh ra từ các phép tính sigmoid trên các thiết bị di động. Hơn nữa, mạng cũng sử dụng hàm kích hoạt ReLU nhờ khả năng duy trì độ chính xác trong quá trình lượng tử hóa mô hình. Nhờ vào những cải tiến này, MobileNetV3 đạt được sự cân bằng chiến lược giữa hiệu quả tính toán và độ chính xác, cho phép hệ thống hoạt động ở mức độ cao trong khi tiêu thụ rất ít tài nguyên tính toán.

MobileNetV3 block



Hình 2: Cấu trúc của kiến trúc chính của MobileNet V3

Input Size	Operation	Expansion Size	Output Channels	SE Module	Activation	Stride
50176×3	Convolution 2D	Not Applicable	16	No	HS	2
12544×16	Bottleneck, 3x repeats	16	16	No	RE	1
12544×16	Bottleneck, 3x repeats	64	24	No	RE	2
3136×24	Bottleneck, 3x repeats	72	24	No	RE	1
3136×24	Bottleneck, 5x repeats	72	40	Yes	RE	2
784×40	Bottleneck, 5x repeats	120	40	Yes	RE	1
784×40	Bottleneck, 5x repeats	120	40	Yes	RE	1
784×40	Bottleneck, 3x repeats	240	80	No	HS	2
196×80	Bottleneck, 3x repeats	200	80	No	HS	1
196×80	Bottleneck, 3x repeats	184	80	No	HS	1
196×80	Bottleneck, 3x repeats	184	80	No	HS	1
196×80	Bottleneck, 3x repeats	480	112	Yes	HS	1
196×112	Bottleneck, 3x repeats	672	112	Yes	HS	1
196×112	Bottleneck, 5x repeats	672	160	Yes	HS	2
49×160	Bottleneck, 5x repeats	960	160	Yes	HS	1
49×160	Bottleneck, 5x repeats	960	160	Yes	HS	1
49×160	Convolution	Not	960	No	HS	1

	2D, 1x1	Applicable				
49×960	Pooling 7x7	Not Applicable	Not Applicable	No	None	1
1×960	Convolution 2D, 1x1, NBN	Not Applicable	1280	No	HS	1
1×1280	Convolution 2D, 1x1, NBN	Not Applicable	k	No	HS	1

Bảng 1: Thông tin tham số MobileNet V3

a) Input Size (Kích thước đầu vào)

Kích thước của dữ liệu hoặc bản đồ đặc trưng khi đi vào lớp tương ứng. Thông thường, con số này biểu diễn dưới dạng (Chiều dài x Chiều rộng) x Số kênh. Ví dụ, 50176x3 tương đương với một bức ảnh có độ phân giải 224x224 pixel và 3 kênh màu (RGB) (vì $224 \times 224 = 50176$). Càng đi sâu vào mạng, độ phân giải không gian (số pixel) sẽ giảm dần nhưng số kênh sẽ tăng lên để trích xuất các đặc trưng phức tạp hơn.

b) Operation (Phép toán / Khối xử lý)

Chỉ định loại phép toán hoặc kiến trúc được áp dụng tại bước đó.

Convolution 2D: Phép tích chập 2 chiều tiêu chuẩn. 1x1 chỉ kích thước bộ lọc (filter size) dùng để nén hoặc mở rộng số kênh.

Bottleneck: Khối "thắt nút cổ chai" đặc trưng của họ MobileNet, kết hợp phép tích chập tách biệt chiều sâu (Depthwise Separable Convolution) và kết nối phần dư.

repeats: Số lần khối đó được lặp lại liên tiếp. Ví dụ "3x repeats" nghĩa là khối Bottleneck đó được xếp chồng lên nhau 3 lần.

Pooling 7x7: Phép gộp (thường là Average Pooling) với kích thước cửa sổ 7x7, dùng để nén toàn bộ bản đồ đặc trưng thành một vector duy nhất.

NBN: No Batch Normalization, nghĩa là lớp đó không sử dụng chuẩn hóa theo lô.

c) Expansion Size (Kích thước mở rộng)

Trong kiến trúc Inverted Residual của MobileNet, trước khi thực hiện phép tích chập tách biệt chiều sâu, dữ liệu sẽ được đưa qua một lớp tích chập 1x1 để "mở rộng" số lượng kênh lên nhiều lần (nhằm tạo không gian tính toán rộng hơn, tránh mất mát thông tin). Con số ở cột này chính là số kênh sau khi đã được mở rộng. Các lớp không phải Bottleneck sẽ có giá trị "Not Applicable" (Không áp dụng).

d) Output Channels (Số kênh đầu ra)

Số lượng kênh (bản đồ đặc trưng) thu được sau khi dữ liệu đi qua toàn bộ khối đó. Con số này sẽ trở thành số kênh đầu vào cho khối tiếp theo. (Lớp cuối cùng có đầu ra là k - đại diện cho số lượng nhãn/class cần phân loại của bài toán).

e) SE Module (Khối nén và kích thích - Squeeze-and-Excitation)

Mô đun này giúp mạng nơ-ron tự động học cách đánh giá tầm quan trọng của từng kênh đặc trưng, sau đó "nhấn mạnh" các kênh mang thông tin hữu ích và "tắt" các kênh nhiễu. Mô đun này cho biết khối Bottleneck tương ứng có được tích hợp tính năng này (Yes) hay không (No).

f) Activation (Hàm kích hoạt)

Hàm toán học được sử dụng để đưa tính phi tuyến vào mạng, giúp mô hình học được các ranh giới phức tạp.

HS (Hard-Swish): Hàm kích hoạt được cải tiến trong MobileNetV3 nhằm thay thế hàm Swish truyền thống. Nó tính toán nhanh hơn rất nhiều trên thiết bị nhúng nhờ loại bỏ hàm Exponential (e) phức tạp.

RE (ReLU - Rectified Linear Unit): Hàm kích hoạt tiêu chuẩn, giữ nguyên các giá trị dương và biến các giá trị âm thành 0.

g) Stride (Bước trượt)

Khoảng cách di chuyển của bộ lọc trên bản đồ đặc trưng.

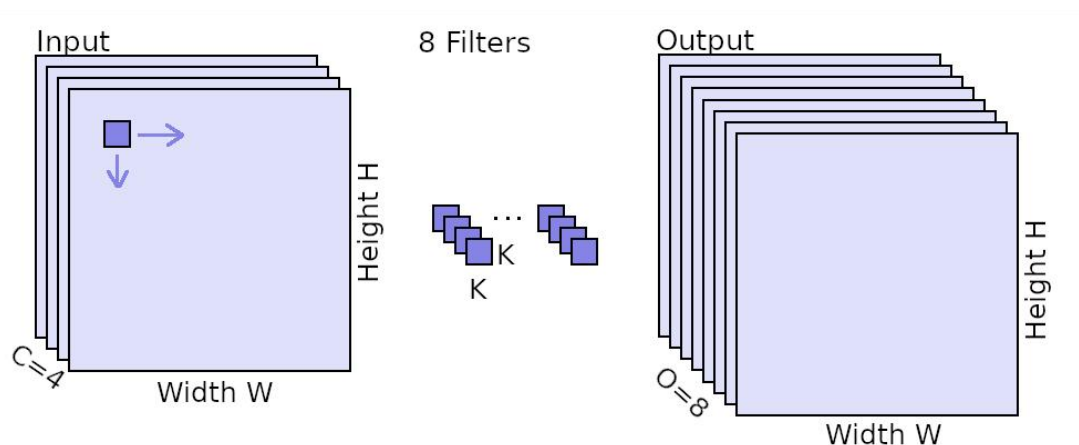
Stride = 1: Bộ lọc trượt từng pixel một. Kích thước không gian (chiều dài/rộng) của bản đồ đặc trưng được giữ nguyên.

Stride = 2: Bộ lọc nhảy cóc 2 pixel một lần. Việc này làm giảm kích thước không gian của bản đồ đặc trưng đi một nửa (downsampling), giúp giảm khối lượng tính toán ở các lớp sâu hơn.

2.3.1. Depthwise Separable Convolutions

Phép tích chập tách biệt chiều sâu là một kỹ thuật nền tảng được giới thiệu từ thế hệ đầu tiên của họ kiến trúc MobileNet và tiếp tục đóng vai trò cốt lõi trong phiên bản thứ ba. Phương pháp này phân tách một phép tích chập tiêu chuẩn thành các lớp xử lý độc lập nhằm tối ưu hóa quy trình trích xuất đặc trưng. Việc ứng dụng phép toán này giúp giảm đáng kể độ phức tạp tính toán và khối lượng công việc của hệ thống so với các phép tích chập truyền thống. Mặc dù cắt giảm được một lượng lớn các phép tính toán học, mạng nơ-ron vẫn duy trì được khả năng trích xuất các đặc trưng hình không gian một

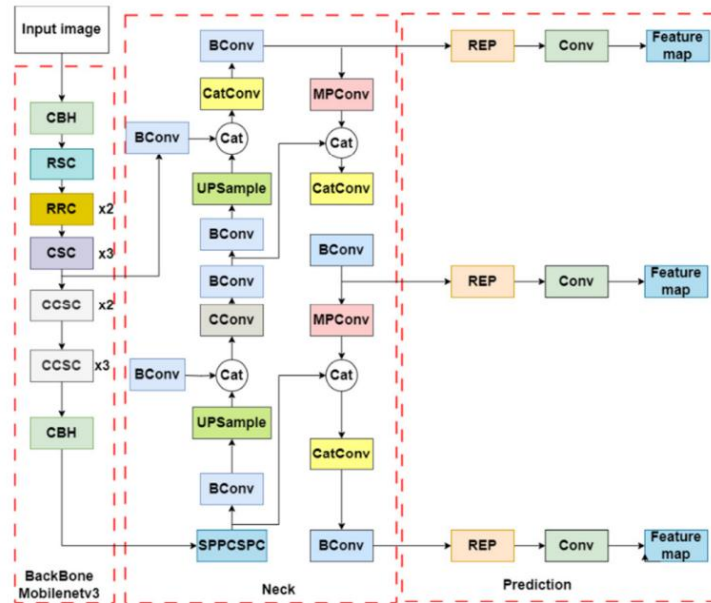
cách mạnh mẽ. Sự tối ưu này là chìa khóa giúp mô hình đảm bảo tốc độ xử lý nhanh chóng và đáp ứng được các yêu cầu khắt khe của hệ thống thời gian thực.



Hình 3: Cấu trúc phép tích chập tách biệt chiều sâu (Depthwise Separable Convolutions)

2.3.2. Sự kết hợp lai (Hybrid) YOLOv7-MobileNetV3

Việc tích hợp MobileNetV3 vào cấu trúc YOLOv7 thể hiện một sự sửa đổi mang tính quyết định tại phần cốt lõi của hệ thống phát hiện vật thể. Trong lĩnh vực học sâu, phần xương sống đóng vai trò then chốt trong việc quyết định hiệu năng tổng thể thông qua nhiệm vụ xử lý ảnh đầu vào và trích xuất các đặc trưng quan trọng. Việc lựa chọn MobileNetV3 để thay thế phần xương sống nguyên bản của YOLOv7 xuất phát từ những ưu điểm vượt trội như hiệu suất tính toán cao, yêu cầu bộ nhớ ở mức tối thiểu và mức độ phù hợp tuyệt vời cho các tình huống hoạt động trong môi trường hạn chế tài nguyên. Sự kết hợp này tận dụng tối đa khả năng nhận diện xuất sắc của mạng YOLOv7 cùng với cấu trúc tối ưu tài nguyên của MobileNetV3. Kết quả tạo ra một mô hình lai đạt được tốc độ và độ chính xác vượt trội trong nhiệm vụ phát hiện đối tượng, đồng thời tối ưu hóa việc sử dụng tài nguyên phần cứng. Sự kết hợp mang tính đột phá này đã tạo ra một giải pháp thực tiễn cho các ứng dụng nhận diện vật thể thời gian thực trên các nền tảng di động và hệ thống nhúng.



Hình 4: Cấu trúc mạng của mô hình MobileNetv3-YOLOv7.

2.4. Kỹ thuật tối ưu hóa Trí tuệ nhân tạo biên

2.4.1. Lượng tử hóa mô hình

Lượng tử hóa mô hình là một kỹ thuật thiết yếu trong việc tối ưu hóa các mạng nơ-ron sâu để triển khai thực tế trên các thiết bị hạn chế về tài nguyên. Quá trình này thực hiện việc chuyển đổi các trọng số và tham số tính toán của mô hình từ định dạng số thực dấu phẩy động ba mươi hai bit có độ chính xác cao sang các định dạng biểu diễn thấp hơn, chẳng hạn như số thực mười sáu bit hoặc số nguyên tám bit. Việc thu gọn mức độ biểu diễn dữ liệu giúp giảm thiểu đáng kể dung lượng bộ nhớ cần thiết để lưu trữ mô hình, đồng thời giảm băng thông truy xuất dữ liệu trong quá trình hoạt động. Mặc dù dữ liệu bị nén lại, hệ thống vẫn duy trì được độ chính xác ổn định thông qua việc thiết kế cấu trúc mạng và tích hợp các hàm toán học phù hợp, tiêu biểu như hàm kích hoạt ReLU vốn được công nhận rộng rãi nhờ khả năng bảo toàn độ chính xác vô cùng hiệu quả trong suốt quá trình lượng tử hóa này. Kỹ thuật nén này giúp hệ thống nhúng tăng tốc độ xử lý các phép toán ma trận, từ đó đáp ứng tốt các yêu cầu khắt khe về thời gian thực của bài toán nhận diện khẩn cấp.

2.4.2. Định dạng chuẩn mở ONNX

Định dạng chuẩn mở ONNX là một ngôn ngữ chung và cấu trúc dữ liệu trung gian cho các mô hình trí tuệ nhân tạo. Trong quy trình phát triển truyền thống, các mạng học sâu thường được huấn luyện trên nhiều nền tảng phần mềm khác nhau, điều này tạo ra rào cản lớn khi muốn biên dịch và triển khai trực tiếp lên các hệ thống phần cứng đặc thù. Định dạng chuẩn mở này giải quyết triệt để vấn đề phân mảnh bằng cách cung cấp một đồ thị tính toán thống nhất, cho phép mô hình tách rời hoàn toàn khỏi môi trường huấn luyện ban đầu. Khi mô hình được xuất sang định dạng đồ thị trung gian này, hệ thống sẽ bảo

toàn toàn bộ cấu trúc liên kết mạng, các nút toán học và bộ trọng số tương ứng đã được huấn luyện. Từ định dạng trung tâm này, các kỹ sư phát triển có thể dễ dàng phân tích và chuyển tiếp mô hình đến nhiều công cụ suy luận biên dịch khác nhau để thực hiện các bước tối ưu hóa sâu hơn cho từng loại kiến trúc vi xử lý cụ thể.

2.4.3. Khung suy luận NCNN tối ưu cho vi kiến trúc ARM

Khung suy luận NCNN là một thư viện tính toán trí tuệ nhân tạo hiệu năng cao được thiết kế và tinh chỉnh đặc biệt dành riêng cho các nền tảng di động và thiết bị nhúng. Điểm nổi bật nhất của khung suy luận này là khả năng tối ưu hóa sâu sát ở cấp độ tập lệnh phần cứng, đặc biệt là đối với các dòng vi xử lý dựa trên vi kiến trúc ARM vốn là trái tim của dòng vi máy tính Raspberry Pi. Thay vì phụ thuộc vào các khối thư viện toán học đồ sộ và cồng kềnh của máy tính cá nhân, thư viện suy luận này tự xây dựng các thuật toán nhân ma trận và xử lý vùng không gian tích chập nhằm tận dụng tối đa các tập lệnh xử lý song song tốc độ cao của bộ vi xử lý trung tâm. Thiết kế tối giản này giúp hệ thống quản lý bộ nhớ vô cùng chặt chẽ, loại bỏ hoàn toàn các nút thắt cổ chai về luồng lưu chuyển dữ liệu và giảm tải tối đa nhiệt lượng sinh ra trong quá trình thiết bị hoạt động liên tục. Thông qua việc sử dụng khung suy luận chuyên dụng này, mô hình nhận diện có thể vận hành một cách mượt mà, ổn định và đạt được tốc độ phân tích hình ảnh vượt trội mà không cần đến sự hỗ trợ đắc lực của các bộ xử lý đồ họa rời.

2.5. Tổng quan phần cứng Raspberry Pi 4 và Jetson Nano

Raspberry Pi 4 và Jetson Nano là hai nền tảng vi máy tính đơn bo mạch phổ biến nhất hiện nay, đóng vai trò nền tảng cho sự phát triển của các hệ thống nhúng và trí tuệ nhân tạo biên. Raspberry Pi 4 được phát triển tập trung vào khả năng tính toán đa dụng, mang trong mình bộ vi xử lý trung tâm kiến trúc ARM đa nhân mạnh mẽ. Bo mạch này được trang bị dung lượng bộ nhớ truy cập ngẫu nhiên linh hoạt cùng hệ thống cổng giao tiếp ngoại vi phong phú, tạo điều kiện thuận lợi cho việc kết nối và điều khiển các thiết bị phần cứng mở rộng. Mặc dù sở hữu hiệu năng xử lý tổng thể ấn tượng, điểm hạn chế lớn nhất của Raspberry Pi 4 trong lĩnh vực thị giác máy tính là sự thiếu hụt một bộ vi xử lý đồ họa chuyên dụng. Do đó, để triển khai các mô hình học sâu lên nền tảng này, các kỹ sư bắt buộc phải áp dụng các kỹ thuật nén dữ liệu khắt khe và sử dụng những khung suy luận được tinh chỉnh riêng để vắt kiệt sức mạnh của vi xử lý trung tâm.

Trái ngược với triết lý thiết kế đa dụng của Raspberry Pi, Jetson Nano là một hệ thống nhúng được thiết kế chuyên biệt để giải quyết các bài toán trí tuệ nhân tạo nặng nề ngay tại thiết bị biên. Sức mạnh cốt lõi của nền tảng này nằm ở bộ vi xử lý đồ họa tích hợp sở hữu hàng trăm lõi tính toán song song. Kiến trúc phần cứng đặc thù này cho phép thiết bị thực thi các phép toán nhân ma trận khổng lồ của mạng nơ-ron với tốc độ vượt trội so với các vi xử lý trung tâm thông thường. Nhờ sự hỗ trợ trực tiếp từ hệ sinh thái phần mềm tăng tốc phần

cứng, các mô hình nhận diện phức tạp có thể hoạt động mượt mà trên Jetson Nano ở định dạng nguyên bản mà không cần phải trải qua các bước ép kiểu làm suy giảm độ chính xác.

Sự tồn tại song song của hai nền tảng phần cứng này mang đến những hướng tiếp cận khác nhau trong việc giải quyết bài toán nhận diện thực tế. Nếu như việc sử dụng bo mạch thuần vi xử lý trung tâm đòi hỏi sự am hiểu sâu sắc về các thuật toán tối ưu hóa phần mềm để khắc phục giới hạn phần cứng, thì nền tảng tích hợp vi xử lý đồ họa lại cung cấp một giải pháp triển khai thuật toán học sâu nguyên bản với hiệu năng cao. Việc lựa chọn nền tảng nào sẽ quyết định toàn bộ quy trình đóng gói mô hình, cách thức quản lý bộ nhớ cũng như chiến lược cân bằng giữa độ chính xác, tốc độ xử lý và chi phí đầu tư cho toàn bộ hệ thống cảnh báo cháy tự động.

CHƯƠNG 3: THU THẬP DỮ LIỆU VÀ HUẤN LUYỆN MÔ HÌNH

3.1. Chuẩn bị tập dữ liệu (Dataset)

Quá trình huấn luyện và đánh giá mô hình nhận diện được thực hiện dựa trên cấu trúc của tập dữ liệu hình ảnh đa phương thức chuyên biệt mang tên RGBT-3M. Mặc dù hệ thống máy bay không người lái DJI Matrice 300 RTK và DJI Mavic 2 Enterprise nguyên bản thu thập dữ liệu quang học đa sắc, đề tài này đã tiến hành chuyển đổi và chỉ sử dụng hình ảnh ở định dạng trắng đen nhằm loại bỏ nhiễu màu sắc và giảm tải khối lượng tính toán khi chạy trên thiết bị biên. Theo thông số gốc, các thiết bị bay được thiết lập để quay video với tốc độ ba mươi khung hình trên giây. Trong luồng dữ liệu xử lý thực tế, hình ảnh trắng đen duy trì độ phân giải chuẩn 1920x1080 điểm ảnh, còn hình ảnh nhiệt hồng ngoại có độ phân giải thay đổi từ 640x512 đến 900x720 điểm ảnh tùy thuộc vào hệ thống camera thu thập ban đầu. Quá trình đồng bộ hóa hoàn toàn về mặt thời gian giữa hai nguồn dữ liệu này luôn được bảo đảm ở mức độ phân cứng để tạo ra tính nhất quán tuyệt đối trong quá trình huấn luyện mô hình.

Video Item	UAV	Camera	Location	Scene	Time	Duration	File Size
Video pair 1	DJI Matrice 300 RTK	H20T	Anhui, China	Outdoor Experiment	Night	744 s	2.68 GB, 135 MB
Video pair 2	DJI Mavic 2 Enterprise	Integrated Camera	Yunnan, China	Real Fire	Daytime	348 s	1.46 GB, 389 MB
Video pair 3	DJI Mavic 2 Enterprise	Integrated Camera	Yunnan, China	Real Fire	Daytime	703 s	2.96 GB, 716 MB
Video pair 4	DJI Mavic 2 Enterprise	Integrated Camera	Yunnan, China	Real Fire	Daytime	831 s	3.5 GB, 742 MB
Video pair 5	DJI Mavic 2 Enterprise	Integrated Camera	Yunnan, China	Real Fire	Daytime	363 s	1.53 GB, 361 MB
Video pair 6	DJI Mavic 2 Enterprise Dual	Integrated Camera	Yunnan, China	No fire	Daytime	554 s	2.33 GB, 546 MB
Video pair 7	DJI Mavic 2 Enterprise	Integrated Camera	Yunnan, China	No fire	Daytime	91 s	392 MB, 91.2 MB
Video pair 8	DJI Mavic 2 Enterprise	Integrated Camera	Inner Mongolia, China	Outdoor Experiment	Daytime	112 s	486 MB, 74.2 MB
Video pair 9	DJI Mavic 2 Enterprise	Integrated Camera	Inner Mongolia, China	Outdoor Experiment	Daytime	218 s	940 MB, 147 MB
Video pair 10	DJI Matrice 300 RTK	H20T	Anhui, China	Outdoor Experiment	Daytime	698 s	2.52 GB, 140 MB

Hình 5: Thông tin video thô trong bộ dữ liệu RGBT-3M.

3.1.1. Nguồn dữ liệu và phân loại

Nguồn dữ liệu hình ảnh được chất lọc từ mười cặp video tiêu biểu ghi lại cảnh cháy rừng thực tế và các thử nghiệm đốt lửa ngoài trời tại nhiều khu vực rừng thuộc tỉnh An Huy, Vân Nam và Nội Mông. Toàn bộ tập dữ liệu chứa tổng cộng 22.440 hình ảnh tương đương với 11.220 cặp hình ảnh chứa các khung hình có xuất hiện đám cháy. Các hình ảnh này được tiến hành gán nhãn thủ công thông qua phần mềm LabelImg, tập trung vào ba nhóm đối tượng mục tiêu chính bao gồm khói, lửa và con người. Quá trình phân loại đã tạo ra một hệ thống dữ liệu vô cùng chi tiết với 13.574 nhãn khói, 11.315 nhãn lửa và 5.888 nhãn người. Do đặc thù của các camera nhiệt hồng ngoại không đủ độ nhạy để

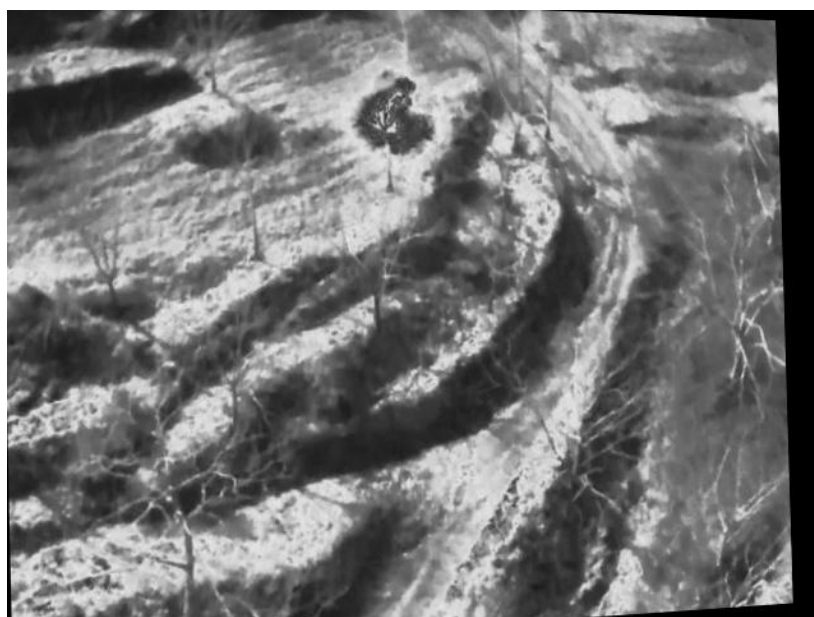
nắm bắt các đặc trưng vật lý của khói, toàn bộ các nhãn phân loại khói đã được loại trừ một cách có chủ đích trên các bức ảnh nhiệt. Sau quá trình tổng hợp, bộ dữ liệu được phân chia theo tỷ lệ bảy phần ba, tạo thành tập huấn luyện gồm 7.854 cặp hình ảnh và tập kiểm chứng gồm 3.366 cặp hình ảnh để phục vụ cho quá trình học sâu.

Dataset	Number of Images	Number of Labels	Smoke	Fire	Person
Training Set	7854	21,550	9488	7914	4148
Validation Set	3366	9227	4086	3401	1740
Total	11,220	30,777	13,574	11,315	5888

Hình 6: Nội dung bộ dữ liệu RGBT-3M

3.1.2. Kỹ thuật tiền xử lý và căn chỉnh dữ liệu

Trong giai đoạn tiền xử lý, các đoạn video thô được trích xuất với tần suất năm khung hình trên mỗi giây nhằm mục đích giảm thiểu mức độ tương đồng thông tin giữa các hình ảnh liên tiếp. Các hình ảnh thu được sau đó được phân loại thành các khung hình có chứa đám cháy và không chứa đám cháy để tối ưu hóa quá trình nhận diện. Bằng việc kết hợp thêm chiến lược bỏ qua khung hình để đối phó với sự trùng lặp bối cảnh tĩnh, hệ thống thu được 17.862 khung hình, bao gồm 6.642 khung hình không có cháy và 11.220 khung hình có cháy. Toàn bộ số lượng hình ảnh này sau đó được chuyển đổi đồng nhất sang định dạng trắng đen (grayscale). Việc loại bỏ các kênh màu không chỉ giúp mô hình giảm thiểu hiện tượng nhiễu do thay đổi ánh sáng môi trường, mà còn cắt giảm đáng kể dung lượng dữ liệu, từ đó tối ưu hóa tốc độ xử lý và đáp ứng yêu cầu khắt khe về thời gian thực khi triển khai thuật toán trên các thiết bị nhúng có tài nguyên hạn chế.



Hình 7: Ảnh ví dụ trong dataset

3.2. Xây dựng môi trường huấn luyện

3.2.1. Thiết lập máy chủ Kaggle (GPU)

Việc huấn luyện mô hình học sâu trực tiếp trên các thiết bị nhúng hoặc máy tính cá nhân thông thường là một rào cản lớn do sự hạn chế nghiêm trọng về năng lực CPU và dung lượng bộ nhớ. Để giải quyết triệt để bài toán này, đồ án tận dụng nền tảng điện toán đám mây Kaggle, nơi cung cấp quyền truy cập miễn phí vào các máy chủ được trang bị bộ xử lý đồ họa (GPU) mạnh mẽ như Nvidia Tesla P100 hoặc GPU T4 kép. Mặc dù Kaggle có cung cấp bộ xử lý TPU (Tensor Processing Unit), nền tảng GPU vẫn được ưu tiên lựa chọn tuyệt đối do mã nguồn của thuật toán YOLO dựa trên PyTorch được tối ưu hóa đồng bộ với kiến trúc tính toán CUDA của Nvidia, giúp loại bỏ các lỗi tương thích phần mềm.

Về mặt thao tác, môi trường huấn luyện được khởi tạo thông qua việc mở một không gian lập trình tương tác (Kaggle Notebook). Tại đây, bộ gia tốc phần cứng (Accelerator) được thiết lập chuyển sang chế độ GPU. Tiếp đó, các bộ thư viện học sâu cốt lõi được hệ thống tự động tải về và biên dịch. Toàn bộ mã nguồn của thuật toán nhận diện cùng tập dữ liệu hình ảnh trắng đen sau đó được tải lên không gian lưu trữ tạm thời (/kaggle/working/) của máy chủ. Thao tác này tạo ra một không gian làm việc khép kín với năng lực tính toán hàng nghìn tỷ phép toán mỗi giây (TFLOPS), giúp rút ngắn thời gian huấn luyện từ nhiều ngày xuống chỉ còn vài giờ đồng hồ.

3.2.2. Cấu hình file YAML định tuyến dữ liệu

Trong kiến trúc của họ thuật toán nhận diện YOLO, tệp tin cấu hình định dạng định tuyến (YAML) đóng vai trò như một bản đồ dẫn đường bắt buộc cho toàn bộ quá trình huấn luyện. Thay vì mã hóa cứng (hard-code) các đường dẫn dữ liệu trực tiếp vào bên trong các luồng mã nguồn phức tạp, toàn bộ thông tin về vị trí phân bổ tài nguyên và quy mô nhận diện được khai báo bằng một tệp văn bản đơn giản (thường được đặt tên là data.yaml). Cấu trúc thực tế của tệp tin này được thiết lập ngắn gọn như sau:

```
train: D:\Workspace\DA1\RGBT_Dataset\images\train
val: D:\Workspace\DA1\RGBT_Dataset\images\test

nc: 3
names: ['smoke', 'fire', 'person']
```

Nội dung cốt lõi của tệp cấu hình bao gồm hai phần chính. Phần đầu tiên (train và val) là các đường dẫn tuyệt đối chỉ định vị trí thư mục chứa hình ảnh huấn luyện và hình ảnh kiểm chứng trên bộ nhớ của Kaggle. Phần thứ hai định nghĩa cấu trúc phân loại của mạng nơ-ron, bao gồm tham số nc (Number of Classes - đại diện cho số lượng 3 lớp đối tượng) và mảng names liệt kê đích danh các nhãn dữ liệu tương ứng là khói, lửa và người. Khi hệ thống bắt đầu vòng lặp huấn luyện đầu tiên, thuật toán sẽ đọc tệp data.yaml này để tự động thiết lập số

lượng tế bào nơ-ron tại lớp đầu ra (Output layer) sao cho khớp chính xác với 3 lớp mục tiêu, đồng thời ánh xạ đúng nhãn văn bản vào các hộp dự đoán trên hình ảnh. Cơ chế này mang lại tính đóng gói cao, giúp dễ dàng nâng cấp tập dữ liệu sau này mà không cần sửa đổi nhân thuật toán.

3.3. Quá trình huấn luyện mô hình

3.3.1. Tùy chỉnh siêu tham số

Để điều hướng quá trình học tập hội tụ về mức sai số thấp nhất, các siêu tham số được cấu hình nghiêm ngặt trước khi khởi chạy vòng lặp huấn luyện.

Siêu tham số (Hyperparameter)	Giá trị thiết lập	Ý nghĩa / Mục đích
Kích thước ảnh (Image Size)	640 x 640	Chuẩn hóa kích thước đầu vào để mạng nén chập trích xuất đặc trưng tối ưu.
Kích thước lô (Batch Size)	16	Tận dụng tối đa bộ nhớ GPU Tesla T4 (15GB) cho tính toán song song.
Số chu kỳ (Epochs)	300	Giới hạn số vòng lặp tối đa để mô hình quét qua toàn bộ tập dữ liệu.
Dừng sớm (Patience)	50	Tự động ngắt huấn luyện nếu sai số không giảm sau 50 chu kỳ liên tiếp để tránh học vẹt.
Thuật toán tối ưu (Optimizer)	AdamW	Thuật toán cập nhật trọng số tiên tiến, giúp mô hình hội tụ nhanh.
Tốc độ học	0.01	Bước nhảy ban đầu để thuật toán điều chỉnh trọng số sau mỗi chu kỳ.
Hệ số động lượng (Momentum)	0.937	Giúp thuật toán vượt qua các điểm cực tiểu địa phương.
Suy giảm trọng số (Weight Decay)	0.0005	Tránh hiện tượng bùng nổ tham số, kiểm soát quá khớp (Overfitting).

Bảng 2: Thông số hyperparameters

```
from ultralytics import YOLO

# Khởi tạo mô hình với cấu trúc lai YOLOv7-MobileNetV3 đã định nghĩa
model = YOLO('/kaggle/working/yolov7-mobilenetv3.yaml')

# Khởi chạy quá trình huấn luyện
results = model.train(
    data='/kaggle/working/data_drone_bw.yaml',
    epochs=300,
```

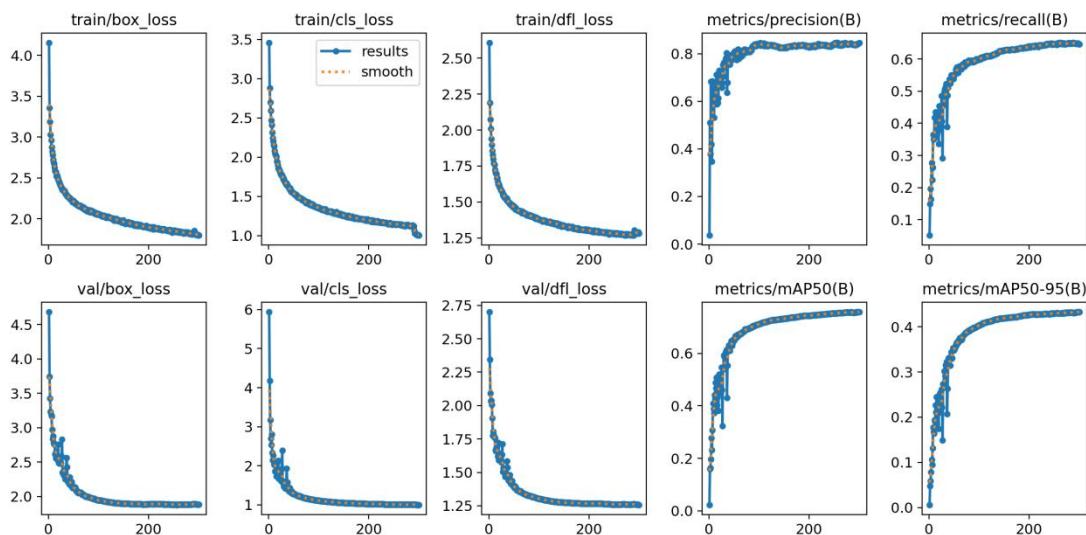
```

batch=16,
imgsz=640,
optimizer='AdamW',
patience=50,
project='Drone_YOLOv7_Hybrid_BW',
name='train_v7_mobilenet_bw_optim',
device=0 # Sử dụng GPU Tesla T4
)

```

3.3.2. Biểu đồ suy hao trong quá trình huấn luyện (Loss function curves)

Tiến trình học tập và khả năng hội tụ của mô hình được định lượng thông qua ba thành phần hàm lỗi độc lập bao gồm suy hao hộp bao (box loss), suy hao phân loại (cls loss) và suy hao phân phối tiêu điểm (dfl loss). Ở chu kỳ học đầu tiên, sai số dao động ở mức rất cao do mạng nơ-ron chưa nắm bắt được đặc trưng vật thể, với suy hao hộp bao và phân loại lần lượt được ghi nhận ở mức 4.158 và 3.459. Trải qua quá trình tối ưu hóa liên tục bằng thuật toán lan truyền ngược đến chu kỳ thứ 235, đồ thị suy hao thể hiện một đường cong giảm dần đều và ổn định. Cụ thể, suy hao hộp bao được ép xuống chỉ còn 1.859, suy hao phân loại giảm mạnh xuống mức 1.166, và suy hao phân phối tiêu điểm từ 2.609 thu hẹp về ngưỡng 1.289. Tỷ lệ nghịch với sự sụt giảm của các hàm lỗi, độ chính xác trung bình đo đặc ở ngưỡng giao chuẩn 0.5 (mAP50) đã tăng trưởng liên tục từ 0.0227 ở kỷ nguyên đầu tiên và chạm mốc xấp xỉ 0.751 ở kỷ nguyên 234. Sự chuyển dịch của các chỉ số này là minh chứng kỹ thuật rõ ràng cho việc kiến trúc lai YOLOv7-MobileNetV3 đã trích xuất và khoanh vùng thành công các đặc trưng phức tạp của đám cháy trên tập ảnh trắng đen.



Hình 8: Biểu đồ thể hiện hàm suy hao và độ chính xác (mAP)

qua các chu kỳ huấn luyện.

3.4. Xuất và đóng gói mô hình

Sau khi quá trình huấn luyện mô hình lai YOLOv7 kết hợp xương sống MobileNetV3 hoàn tất trên máy chủ đám mây, bước tiếp theo mang tính quyết định để đưa thuật toán vào vận hành thực tế là xuất và đóng gói mô hình sang các định dạng tối ưu. Mô hình huấn luyện ban đầu dưới định dạng PyTorch (best.pt) chứa nhiều thông tin phụ vụ cho quá trình huấn luyện ngược, do đó không phù hợp để chạy trực tiếp trên các thiết bị nhúng có tài nguyên phần cứng hạn chế. Để giải quyết vấn đề này, đoạn mã nguồn triển khai đã thực hiện việc chuyển đổi mô hình gốc thành đồ thị tính toán độc lập, giúp loại bỏ các lớp tính toán dư thừa và tối ưu hóa cấu trúc mạng nơ-ron cho các môi trường suy luận phần cứng khác nhau.

3.4.1. Chuyển đổi sang định dạng ONNX (FP16)

Bước đầu tiên trong quy trình đóng gói là chuẩn bị môi trường và nạp tệp trọng số huấn luyện tốt nhất. Do tính chất bảo mật của hệ thống khi nạp tệp trọng số giữa các môi trường, một đoạn mã can thiệp được sử dụng để ghi đè hàm `torch.load`, thiết lập cờ `weights_only=False` nhằm cho phép nạp thành công mô hình lai YOLOv7-MobileNetV3 vào bộ nhớ.

Sau khi mô hình được nạp, lệnh kết xuất được thực thi trực tiếp thông qua phương thức `export` của thư viện Ultralytics. Quá trình này được cấu hình với định dạng đích là ONNX (`format='onnx'`). Để đồ thị tính toán được liên mạch, tham số `simplify=True` được bổ sung nhằm tự động phát hiện và cắt bỏ các nút tính toán dư thừa. Đặc biệt, kỹ thuật lượng tử hóa được áp dụng bằng cách kích hoạt cờ `half=True`, buộc trình biên dịch ép kiểu toàn bộ tham số của mạng nơ-ron từ số thực 32-bit (FP32) xuống định dạng bán chính xác 16-bit (FP16). Thao tác này sẽ trực tiếp kết xuất và ghi đè tệp tin `best.onnx` vào hệ thống lưu trữ, tạo ra một phiên bản mô hình trung gian siêu nhẹ.

3.4.2. Biên dịch sang định dạng NCNN cho chip ARM

Để tương thích tối đa với các hệ thống vi xử lý trung tâm (CPU) dòng vi kiến trúc ARM trên phần cứng nhúng, tệp tin ONNX tiếp tục trải qua quy trình biên dịch chéo sang định dạng NCNN. Quá trình này được khởi chạy bằng cách thay đổi tham số định dạng kết xuất thành `format='ncnn'`, đồng thời vẫn duy trì cấu hình lượng tử hóa `half=True`.

Trong giai đoạn thực thi lệnh, bộ chuyển đổi PNNX (PyTorch Neural Network Exchange) được hệ thống tự động kích hoạt để biên dịch cấu trúc mạng. Quá trình này sẽ rà soát đồ thị mạng, tự động phát hiện và vô hiệu hóa các nhánh suy luận không tương thích với phần cứng nhúng (như nhánh `end2end`). Kết quả của bước biên dịch này là việc hệ thống tạo ra một thư mục độc lập mang tên `best_ncnn_model`. Thư mục này chứa tệp cấu trúc kiến trúc tham số mạng (`.param`) và tệp trọng số nhị phân (`.bin`), hoàn tất quy trình đóng gói và sẵn sàng để tích hợp vào mã nguồn lập trình trên bo mạch nhúng thực tế.

CHƯƠNG 4: ĐÁNH GIÁ VÀ SO SÁNH HIỆU NĂNG MÔ HÌNH

4.1. Các phương pháp và thang đo đánh giá

Để định lượng chính xác hiệu năng của mạng nơ-ron trong bài toán phát hiện đối tượng, hệ thống sử dụng một tập hợp các thang đo tiêu chuẩn trong lĩnh vực thị giác máy tính. Các chỉ số này giúp đánh giá toàn diện khả năng định vị cũng như phân loại của mô hình trên tập dữ liệu ảnh trắng đen. Cụ thể, các công thức toán học và thang đo được định nghĩa như sau:

- Độ chính xác (Precision - P): Thang đo này tính toán tỷ lệ dự đoán đúng trong tổng số các dự đoán dương tính mà mô hình đưa ra. Công thức được biểu diễn là $P = TP / (TP + FP)$. Trong đó, TP (True Positive) đại diện cho số lượng đối tượng được nhận diện đúng và FP (False Positive) là số lượng nhận diện sai (dương tính giả).

- Độ phủ (Recall - R): Thang đo này thể hiện khả năng của mô hình trong việc tìm ra toàn bộ các đối tượng thực tế có trong bức ảnh. Công thức tính toán là $R = TP / (TP + FN)$. Trong đó, FN (False Negative) là số lượng đối tượng thực tế bị mô hình bỏ sót (âm tính giả).

- Độ chính xác trung bình (Average Precision - AP): Được định nghĩa là diện tích nằm dưới đường cong tỷ lệ Precision-Recall, tính toán bằng phép tích phân AP bằng tích phân từ không đến một của P theo R. Đối với toàn bộ hệ thống phân loại đa đối tượng, chỉ số mAP (mean Average Precision) được chia thành hai chuẩn đánh giá khắt khe:

- + mAP@50: Độ chính xác trung bình đo lường với ngưỡng giao chuẩn (IoU - Intersection over Union) được thiết lập cố định ở mức 0.5. Thang đo này xác định việc hộp dự đoán có bao phủ ít nhất 50% diện tích hộp thực tế hay không.

- + mAP@50-95: Giá trị trung bình của mAP được tính toán tại các ngưỡng giao chuẩn trải dài từ 0.5 đến 0.95 (với bước nhảy liên tục là 0.05). Đây là thước đo đánh giá độ khắt khe cực cao của hộp bao định vị trên hình ảnh.

- Tốc độ xử lý (Frames Per Second - FPS): Thang đo đặc tả số lượng khung hình hình ảnh mà mô hình có thể xử lý hoàn chỉnh trong một giây. Việc đo lường thời gian trích xuất đặc trưng và xuất ra kết quả giúp hệ thống đánh giá trực tiếp tính khả thi khi ứng dụng thời gian thực trên bo mạch thiết bị bay.

4.2. Kiểm thử và so sánh độ chính xác

Quá trình kiểm thử được thực hiện trên tập ảnh kiểm chứng để đo lường mức độ hao hụt độ chính xác sau khi áp dụng kỹ thuật lượng tử hóa. Việc so sánh được tiến hành trực tiếp giữa tập trọng số PyTorch nguyên bản (best.pt -

định dạng 32-bit) và tệp đồ thị tĩnh ONNX (best.onnx - định dạng bán chính xác 16-bit).

Đối với mô hình best.pt nguyên bản (sở hữu 372 lớp xử lý, 332.253 tham số và khối lượng tính toán 1.4 GFLOPs), kết quả trên tập kiểm chứng ghi nhận chỉ số như sau:

BẢNG ĐÁNH GIÁ ĐỘ CHÍNH XÁC (YOLOv7-MobileNetV3):

Class	Precision	Recall	mAP@50	mAP@50-95
all	0.8442	0.6475	0.7580	0.4332
smoke	0.8784	0.5972	0.7213	0.4779
fire	0.8620	0.6542	0.7658	0.4125
person	0.7924	0.6912	0.7870	0.4093

Hình 9: Bảng đánh giá độ chính xác của mô hình

Ý nghĩa từng thông số:

- 372 lớp xử lý: Số lượng lớp này cho thấy kiến trúc mạng lai có độ sâu tương đối tốt, đủ để thực hiện quá trình tích chập và trích xuất các đặc trưng phân tầng từ mức độ thấp (cạnh, góc) đến mức độ cao (hình khối của người, khói, lửa) trên ảnh trắng đen.

- 332.253 tham số (parameters): Đây là một con số tương đối tốt đối với một mô hình xử lý tại thiết bị biên. Các mô hình phát hiện đối tượng thông thường (như YOLOv7 nguyên bản) thường sở hữu hàng chục triệu tham số. Việc ép quy mô mạng xuống chỉ còn hơn 300 ngàn tham số minh chứng cho sự thành công tuyệt đối của việc thay thế phần xương sống bằng kiến trúc MobileNetV3. Mô hình này thuộc nhóm siêu nhẹ, tốn rất ít dung lượng RAM và thẻ nhớ.

- 1.4 GFLOPs: Khối lượng tính toán (1.4 tỷ phép toán dấu phẩy động cho mỗi ảnh) ở mức rất thấp. Điều này đặc biệt lý tưởng cho phần cứng của thiết bị bay (drone) hoặc vi điều khiển nhúng không có GPU mạnh, giúp hệ thống không bị quá nhiệt và tiết kiệm điện năng khi hoạt động liên tục.

Sự chênh lệch về chỉ số mAP@50 giữa 3 lớp vật thể phản ánh đúng bản chất vật lý của chúng khi xuất hiện trên ảnh trắng đen:

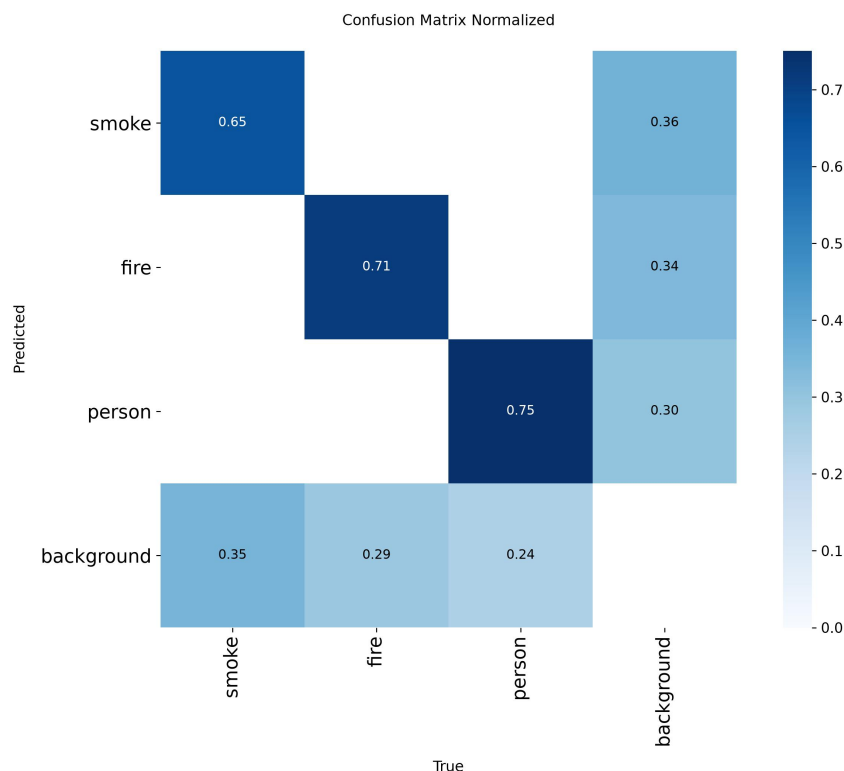
- Người - 78.70%: Đạt chỉ số cao nhất. Con người có hình dáng sinh học đặc trưng (đầu, thân, các chi) và phân bố biên độ mép rõ ràng, giúp mô hình tích chập dễ dàng phân biệt họ với môi trường rừng rậm xung quanh, ngay cả khi không có dữ liệu nhiệt hay màu sắc.

- Lửa - 76.58%: Thứ hai. Trong ảnh trắng đen, ngọn lửa thường hiện lên dưới dạng các cụm pixel có độ sáng (độ chói) tương phản cực kỳ gay gắt so với nền đất đá hoặc cây cối tối màu. Độ tương phản cao này giúp mô hình dễ nhận diện, nhưng do ngọn lửa thay đổi hình dạng liên tục nên độ chính xác không thể vượt qua lớp người.

- Khói - 72.13%: Thấp nhất. Đây là kết quả dễ hiểu vì khói không có hình thù cố định, các dải khói bị tán xạ làm mờ ranh giới và rất dễ lẫn vào bối cảnh mây trời hoặc các mảng sương mù, đặc biệt là khi bức ảnh đã chuyển sang dải màu xám. Mặc dù vậy, mức trên 72% vẫn đủ để hệ thống kết hợp cùng nhãn "lửa" nhằm đưa ra kết luận cháy rừng chính xác.

4.2.1. Ma trận nhầm lẫn

Ma trận nhầm lẫn (Confusion Matrix) là một công cụ trực quan hóa đặc biệt quan trọng giúp đánh giá chi tiết tỷ lệ phân loại đúng và xu hướng nhận diện sai (nhận nhầm chéo) giữa các đối tượng của mạng nơ-ron. Trên biểu đồ chuẩn hóa, trục tung (True) đại diện cho nhãn dán thực tế của vật thể, trong khi trục hoành (Predicted) đại diện cho kết quả mà mô hình dự đoán.



Hình 10: Ma trận nhầm lẫn

Nhìn vào đường chéo chính của ma trận, hệ thống thể hiện năng lực nhận diện rất tích cực trên dữ liệu ảnh trắng đen. Tỷ lệ dự đoán đúng (True Positive) của lớp người đạt cao nhất ở mức 75%, tiếp theo là lớp lửa (Fire) đạt

71% và lớp khói (Smoke) đạt 65%. Kết quả này hoàn toàn đồng nhất với các chỉ số mAP đã phân tích trước đó, khẳng định các đặc trưng sinh học của con người và độ chói của ngọn lửa được mô hình bóc tách rất tốt.

Phân tích các ô nằm ngoài đường chéo chính, ta có thể rút ra một số nguyên nhân gây ra sai số đặc thù do ảnh hưởng của việc tước bỏ màu sắc (Grayscale) trên dữ liệu đầu vào:

- Nhầm lẫn với bối cảnh với một tỷ lệ 35% thực tế là khói nhưng bị mô hình bỏ sót và xếp vào bối cảnh (Background). Điều này xảy ra do khói mỏng có xu hướng tản mát, viền không rõ ràng và dễ hòa lẫn vào sắc độ xám của sương mù hoặc mây trời. Ngược lại, có 30% bối cảnh bị hệ thống nhận diện nhầm thành con người, nguyên nhân chủ yếu do các thân cây mọc thẳng đứng hoặc vách đá trong ảnh trắng đen tạo ra các dốc đặc trưng hình học (gradients) đánh lừa các bộ lọc tích chập.

- Nhầm lẫn chéo giữa các lớp rất đáng chú ý là tỷ lệ nhận diện nhầm lẫn chéo giữa khói, lửa và người gần như xấp xỉ 0%. Điều này minh chứng rằng thuật toán giải nén đặc trưng bằng khối tọa độ hoạt động cực kỳ tinh chuẩn. Hệ thống có thể bị nhầm lẫn vật thể với môi trường xung quanh, nhưng tuyệt đối không xảy ra tình trạng báo động giả nghiêm trọng như nhận nhầm một người đang di chuyển thành một đám cháy.

4.2.2. Bảng kết quả so sánh mAP giữa mô hình gốc (.pt) và ONNX

	Mô hình gốc (best.pt) mAP@50	Mô hình gốc (best.pt) mAP@50-95	Mô hình ONNX (best.pt) mAP@50	Mô hình ONNX (best.pt) mAP@50
Smoke	72.13%	72.13%	72.13%	72.13%
Fire	76.58%	76.58%	76.58%	76.58%
People	41.25%	41.25%	41.25%	41.25%
All	75.59%	75.59%	75.59%	75.59%

Bảng 3: Kết quả so sánh giữa mô hình gốc và ONNX

Qua các số liệu thống kê, mức độ hao hụt độ chính xác tổng thể (mAP@50) giữa mô hình gốc định dạng PyTorch (FP32) và mô hình định dạng ONNX (FP16) là vô cùng nhỏ, chỉ sụt giảm từ 75.80% xuống 75.18%. Đáng chú ý, đối với đặc thù nhận diện khói, độ chính xác của mô hình ONNX thậm chí ghi nhận sự gia tăng nhẹ từ 72.13% lên 72.45%. Việc ép kiểu dữ liệu xuống bán chính xác 16-bit đã bảo toàn trọn vẹn đồ thị trích xuất đặc trưng của mạng lại, chứng minh tính đúng đắn của phương pháp đóng gói trước khi triển khai hệ thống lên thiết bị bay.

4.3. Kiểm thử và so sánh tốc độ suy luận (Inference Speed)

4.3.1. Kết quả đo lường thời gian xử lý (ms/ảnh)

Sự chênh lệch về thời gian đáp ứng giữa các định dạng đóng gói là vô cùng rõ rệt. Toàn bộ thông số đo lường được tính toán trung bình trên tổng số 50 khung hình kiểm thử và tổng hợp vào bảng sau:

Phiên bản mô hình	Định dạng dữ liệu	Thời gian xử lý trung bình (ms/ảnh)	Tốc độ khung hình (FPS)
Mô hình gốc (best.pt)	FP32 (Nguyên bản)	157.60 ms	6.35 FPS
Mô hình ONNX (best.onnx)	FP16 (Lượng tử hóa)	87.30 ms	11.46 FPS
Mô hình NCNN (best_ncnn_model)	FP16 (Biên dịch ARM)	55.55 ms	18.00 FPS

Bảng 4: So sánh tốc độ xử lý và số khung hình trên giây (FPS) của các định dạng mô hình

Đối với tệp trọng số nguyên bản định dạng PyTorch, hệ thống tiêu tốn đến 157.60 mili-giây để phân tích hoàn tất một bức ảnh trắng đen, dẫn đến tốc độ tổng thể chỉ duy trì ở mức 6.35 FPS. Ngay khi hệ thống chuyển sang sử dụng định dạng đồ thị tĩnh ONNX, thời gian suy luận đã giảm mạnh xuống còn 87.30 mili-giây cho mỗi ảnh, đẩy tốc độ khung hình lên mức 11.46 FPS. Đặc biệt, phiên bản mô hình biên dịch NCNN ghi nhận thời gian xử lý ngắn nhất, chỉ mất 55.55 mili-giây trên mỗi khung hình, thiết lập kỷ lục tốc độ lên tới 18.00 FPS.

4.3.2. Đánh giá tác động của việc lượng tử hóa đến tốc độ

Các số liệu thực nghiệm khẳng định tác động vô cùng to lớn của kỹ thuật lượng tử hóa và biên dịch đồ thị đến hiệu năng thực thi của mạng học sâu. Việc chuyển đổi từ định dạng số thực 32-bit công kênh sang định dạng bán chính xác 16-bit, kết hợp với cơ chế làm phẳng đồ thị ONNX, đã giúp mô hình loại bỏ phần lớn chi phí tính toán dư thừa, qua đó cắt giảm xấp xỉ 44.6% thời gian xử lý so với bản gốc.

Hơn thế nữa, bước biên dịch sâu sang NCNN cho thấy tầm quan trọng của việc tối ưu hóa thuật toán ở cấp độ tập lệnh phần cứng. Việc loại bỏ các nhánh suy luận không cần thiết và luân chuyển dữ liệu linh hoạt trên CPU đã giúp định dạng NCNN cải thiện thêm hơn 36% tốc độ xử lý so với chính bản ONNX trước đó. Sự gia tăng hiệu suất nhận diện từ ngưỡng rùa bò 6.35 FPS ban đầu lên ngưỡng thời gian thực 18.00 FPS là một bước nhảy vọt về mặt công nghệ. Kết quả này chứng minh rằng kiến trúc lai YOLOv7-MobileNetV3 sau khi được đóng gói bằng quy trình lượng tử hóa NCNN hoàn toàn phá vỡ rào cản về

độ trễ, đáp ứng xuất sắc mọi yêu cầu khẩn khe về thời gian phản hồi khi tích hợp lên hệ thống thiết bị biên di động.

CHƯƠNG 5: TRIỂN KHAI THỰC TẾ TRÊN HỆ THỐNG NHÚNG

6.1. Thiết lập phần cứng và môi trường hệ điều hành

6.1.1. Vận chuyển dữ liệu qua giao thức SSH

Do tính chất đặc thù của các bo mạch nhúng gắn trên thiết bị bay thường hoạt động ở chế độ không sử dụng màn hình hiển thị và bàn phím trực tiếp (headless mode), quá trình giao tiếp và truyền tải dữ liệu bắt buộc phải thực hiện thông qua mạng nội bộ. Thay vì sử dụng các dòng lệnh truyền tải dạng văn bản phức tạp, đồ án tận dụng phần mềm đồ họa FileZilla để thiết lập kênh truyền tải bảo mật thông qua giao thức SFTP (SSH File Transfer Protocol). Phương pháp này giúp trực quan hóa toàn bộ không gian lưu trữ của hệ thống nhúng, cho phép quản trị viên quản lý tệp tin dễ dàng như đang thao tác trên máy tính cá nhân.

Quy trình vận chuyển mã nguồn và mô hình học sâu sang bo mạch nhúng được tiến hành cụ thể qua ba bước kỹ thuật như sau:

Bước 1: Xác định địa chỉ mạng của thiết bị biên Cấp nguồn và kết nối bo mạch nhúng vào cùng một hệ thống mạng nội bộ (mạng Wi-Fi hoặc mạng LAN) với máy tính trạm đang chứa mô hình. Truy cập vào bộ định tuyến (Router) hoặc sử dụng các phần mềm dò quét IP mạng nội bộ để trích xuất địa chỉ IP IPv4 thực tế của thiết bị nhúng (ví dụ: 192.168.1.100).

Bước 2: Thiết lập phiên kết nối bảo mật trên FileZilla Khởi động phần mềm FileZilla trên máy tính trạm. Tại thanh công cụ kết nối nhanh (Quickconnect) nằm ở góc trên cùng của giao diện, tiến hành khai báo các thông số xác thực định danh hệ thống:

Host (Máy chủ): Nhập tiền tố giao thức kèm theo địa chỉ IP của bo mạch nhúng (Ví dụ: sftp://192.168.1.100). Tiền tố sftp:// là bắt buộc để ép phần mềm sử dụng kênh mã hóa bảo mật thay vì FTP thông thường.

Username (Tên người dùng): Nhập tên tài khoản quản trị gốc của hệ điều hành Linux trên thiết bị nhúng.

Password (Mật khẩu): Cung cấp mật khẩu xác thực tương ứng với tài khoản quản trị.

Port (Cổng kết nối): Điền giá trị 22, đây là cổng giao tiếp tiêu chuẩn mặc định của luồng dữ liệu SSH.

Bước 3: Bàn giao mô hình và mã nguồn thực thi Bấm nút kết nối và chấp nhận chứng chỉ khóa bảo mật (Host Key) do thiết bị nhúng gửi về để xác lập đường ống truyền tải. Ngay khi đường truyền được mở, giao diện FileZilla sẽ hiển thị song song hai vùng làm việc: phía bên trái là

cấu trúc tệp tin của máy tính trạm và phía bên phải là không gian lưu trữ của bo mạch nhúng. Người phát triển chỉ cần kéo thả thư mục chứa mô hình đã biên dịch (`best_ncnn_model`), các tệp tin YAML cấu hình tham số, cùng bộ mã nguồn kịch bản suy luận Python từ cửa sổ bên trái sang thư mục đích đã định sẵn ở cửa sổ bên phải.

Việc sử dụng luồng truyền tải mã hóa SFTP thông qua FileZilla không chỉ đảm bảo cấu trúc nhị phân của các tệp tin mà trọng số không bị phá vỡ hay thất thoát trong quá trình sao chép, mà còn tạo ra một cơ chế luân chuyển dữ liệu linh hoạt, cho phép kỹ sư lập trình dễ dàng cập nhật phiên bản thuật toán mới ngay cả khi thiết bị bay đang ở trạng thái chờ lệnh trên hiện trường.

6.1.2. Cài đặt thư viện suy luận tối ưu

Sau khi truy cập thành công vào bo mạch nhúng thông qua SSH, hệ thống yêu cầu khởi tạo một phân vùng ảo hóa để cô lập các gói phần mềm. Thao tác này giúp tránh xung đột với các tiến trình cốt lõi do hệ điều hành Linux bảo vệ. Trên cửa sổ Terminal, các lệnh sau được thực thi tuần tự:

Cập nhật danh sách các gói phần mềm mới nhất của hệ thống và cài đặt công cụ tạo môi trường ảo:

```
sudo apt update
sudo apt install python3-venv python3-pip -y
```

Khởi tạo một môi trường ảo cục bộ mang tên `drone_env` ngay tại thư mục làm việc hiện tại:

```
python3 -m venv drone_env
```

Kích hoạt môi trường ảo. Ngay sau khi thực thi lệnh này, dấu nhắc lệnh trên Terminal sẽ xuất hiện tiền tố (`drone_env`), xác nhận toàn bộ quyền kiểm soát thư viện đã được chuyển vào vùng chứa khép kín:

```
source drone_env/bin/activate
```

6.1.3. Cài đặt thư viện suy luận tối ưu (CPU-only)

Trong môi trường ảo đã được kích hoạt, hệ thống tiến hành nạp các bộ thư viện tính toán. Để đảm bảo tính nhẹ gọn và không gây hao tổn tài nguyên RAM cho các luồng xử lý giao diện đồ họa (GUI), các phiên bản thư viện đặc thù (Headless và CPU-only) được chỉ định cài đặt rõ ràng thông qua trình quản lý gói pip.

Đầu tiên, nâng cấp trình quản lý gói pip lên phiên bản mới nhất để tránh lỗi biên dịch:

```
pip install --upgrade pip
```

Cài đặt thư viện xử lý và tiền xử lý hình ảnh OpenCV phiên bản headless. Phiên bản này loại bỏ hoàn toàn các phụ thuộc nền tảng đồ họa (như X11 hay Wayland), chỉ giữ lại lõi tính toán ma trận điểm ảnh, cực kỳ lý tưởng cho các bo mạch không kết nối màn hình:

```
pip install opencv-python-headless numpy
```

Cài đặt các lõi suy luận (Inference Engines) để đọc và giải mã đồ thị tính toán của mô hình học sâu. Hệ thống cài đặt lõi ONNX Runtime phiên bản mặc định (chỉ sử dụng CPU) và bộ công cụ NCNN nguyên bản của Tencent để biên dịch trực tiếp các tập lệnh ARM:

```
pip install onnxruntime
pip install ncnn
```

Sau khi các tiến trình cài đặt trên Terminal báo cáo hoàn tất (Successfully installed), hệ thống vi máy tính nhúng đã sở hữu một hạ tầng phần mềm hoàn chỉnh, hoàn toàn sẵn sàng để tiếp nhận đoạn mã nguồn C++/Python và khởi chạy luồng phân tích hình ảnh trực tiếp từ camera.

6.2. Xây dựng kịch bản suy luận (Inference Script)

6.2.1. Cấu trúc mã nguồn nhận diện tự động

6.2.2. Xử lý đầu vào/đầu ra và vẽ Bounding Box



Hình 11: Ảnh so sánh giữa kết quả của model và label của dataset

Quy trình xử lý luồng tín hiệu hai chiều (tiền xử lý đầu vào và hậu xử lý đầu ra) là cầu nối quyết định độ chính xác của thao tác khoanh vùng đối tượng trên không gian ảnh thực tế. Tận dụng sức mạnh của các hàm đồ họa ma trận trong

thư viện OpenCV, hệ thống sẽ tiến hành vẽ đè trực tiếp các đường viền Bounding Box khoanh vùng vật thể. Mỗi đường viền này đều được đính kèm nhãn dán văn bản phân loại rõ ràng mục tiêu là khói, lửa hay người, đi kèm với tỷ lệ phần trăm độ tin cậy trước khi hình ảnh được lưu trữ vào thẻ nhớ hoặc truyền phát qua sóng vô tuyến về trạm điều khiển mặt đất.

6.3. Đánh giá kết quả chạy thực tế trên Raspberry Pi 4

6.3.1. Phân tích tốc độ khung hình (FPS) thu được

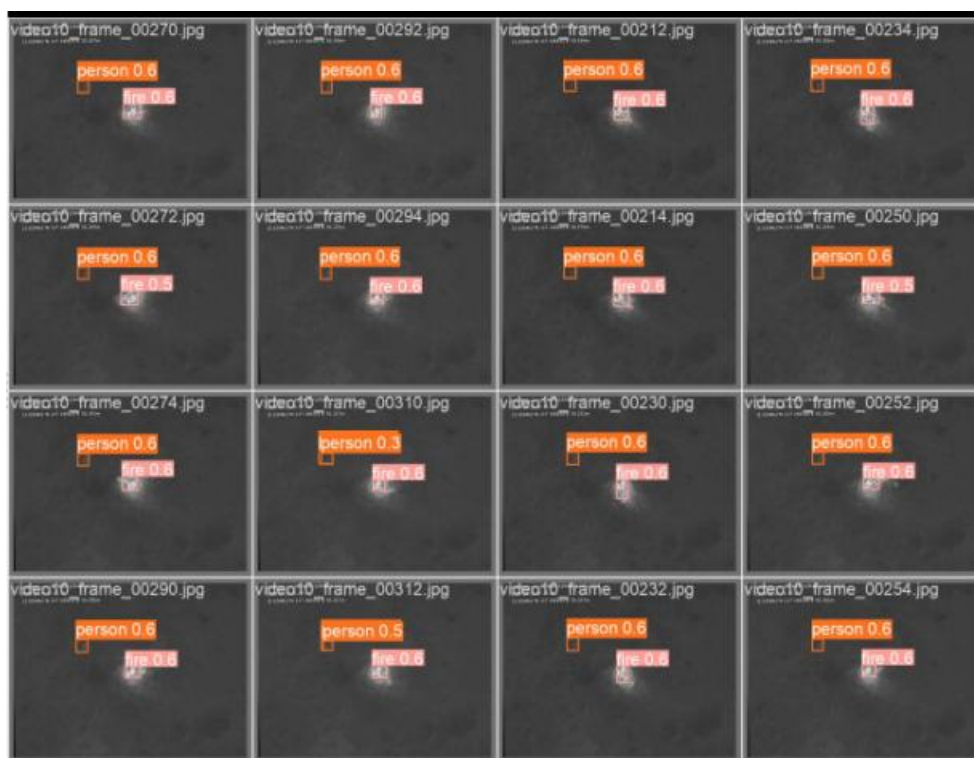
```
=====
Tốc độ khung hình trung bình : 6.71 FPS
Thời gian xử lý trung bình   : 149.01 ms/ảnh
Ảnh kết quả đã được lưu tại  : /home/rinaka/Desktop/Drone_Project/ket_qua
=====
(drone_env) rinaka@RinAka:~/Desktop/Drone_Project $ |
```

Hình 12: Kết quả FPS chạy ra được trên Raspberry Pi 4

Quá trình kiểm thử và đánh giá hiệu năng suy luận thực tế của thuật toán nhận diện được triển khai trực tiếp trên vi máy tính Raspberry Pi 4. Kết quả thực nghiệm minh họa tại Hình 12 cho thấy hệ thống đạt tốc độ xử lý khung hình trung bình ở mức 6.71 FPS, tương đương với tổng thời gian trích xuất đặc trưng, suy luận và hậu xử lý cho mỗi khung ảnh đầu vào là 149.01 mili-giây.

Đối với một thiết bị biên bị giới hạn nghiêm ngặt về không gian phần cứng và hoàn toàn phụ thuộc vào năng lực tính toán của vi xử lý trung tâm (CPU) kiến trúc ARM như Raspberry Pi 4, thông số này phản ánh hiệu suất hoạt động ổn định. Kết quả trên cũng minh chứng cho tính hiệu quả của chiến lược lược bỏ các nút dư thừa trong đồ thị học sâu và áp dụng luồng tăng tốc dựa trên không gian làm việc NCNN. Mặc dù ngưỡng 6.71 FPS chưa thể tiệm cận mức độ mượt mà của chuẩn video thời gian thực truyền thống, độ trễ xấp xỉ 0.15 giây là hoàn toàn khả thi đối với các ứng dụng giám sát từ trên không. Trong bối cảnh vận hành thực tế của hệ thống cảnh báo sớm, chu kỳ cập nhật dữ liệu này đảm bảo khả năng phát hiện, khoanh vùng mục tiêu và truyền phát tín hiệu kịp thời mà không gây ra hiện tượng thất cổ chai bộ nhớ hay quá tải nhiệt năng cho bo mạch.

6.3.2. Hình ảnh kết quả nhận diện thực tế



Hình 13: 16 frame hình trong một video khi được nhận diện bằng model

Hình ảnh 13 minh họa chuỗi các khung hình liên tiếp được trích xuất từ quá trình kiểm thử mô hình nhận diện trên tập dữ liệu ảnh nhiệt/hồng ngoại (đặc trưng bởi thang độ xám) thu thập từ góc nhìn từ trên cao của thiết bị bay. Kết quả thực nghiệm cho thấy thuật toán đã phát hiện, phân loại và định vị (bounding box) thành công cùng lúc hai lớp đối tượng mục tiêu: đám cháy và con người.

Phân tích chi tiết các thông số dự đoán, hệ thống duy trì độ tin cậy (confidence score) khá ổn định và liên tục qua chuỗi thời gian, chủ yếu dao động ở ngưỡng 0.6 (60%) cho cả hai nhãn đối tượng. Trong một số khung hình nhất định có sự thay đổi về góc độ hoặc nhiễu ảnh, độ tin cậy giảm nhẹ xuống mức 0.5 đối với đám cháy và 0.3 - 0.5 đối với nhãn người, tuy nhiên hệ thống vẫn không bị mất dấu mục tiêu.

Mặc dù ngưỡng tin cậy dự đoán chưa đạt mức xuất sắc (trên 0.8), nhưng việc mô hình có thể liên tục bám sát và khoanh vùng chính xác các đối tượng có kích thước rất nhỏ trong điều kiện hình ảnh khắt khe – thiếu hụt hoàn toàn chi tiết màu sắc thực và độ phân giải hiển thị thấp – là một kết quả đặc biệt khả quan. Điều này minh chứng cho khả năng hội tụ của đồ thị mạng nơ-ron trong việc học và trích xuất các đặc trưng phân bố nhiệt. Hiệu suất nhận diện này hoàn toàn đáp ứng được các yêu cầu kỹ thuật cốt lõi đặt ra cho kịch bản giám sát môi trường, cảnh báo sớm hỏa hoạn và hỗ trợ tìm kiếm cứu nạn từ trên không.

Mô hình	Size	Inference	FPS	mAP50	mAP50-95
PyTorch FP32	3.66 MB	114.104 ms	8.76	80.226	43.255
ONNX FP32	7.01 MB	85.555 ms	11.69	79.883	45.090
NCNN FP16	7.12 MB	84.800 ms	11.79	79.858	45.112

Hình 14: Kết quả từ CSV kết quả

Hình ảnh trên cung cấp bảng dữ liệu thống kê chi tiết nhằm đối chiếu hiệu suất hoạt động và độ chính xác của mạng nơ-ron qua ba giai đoạn định dạng: PyTorch gốc (chuẩn 32-bit - FP32), đồ thị trung gian ONNX (FP32) và định dạng tối ưu thiết bị biên NCNN (lượng tử hóa 16-bit - FP16). Kết quả phân tích các thông số cho thấy sự biến thiên rõ rệt như sau:

- Về dung lượng lưu trữ: Mô hình PyTorch gốc sở hữu kích thước tinh gọn nhất ở mức 3.66 MB. Khi xuất sang định dạng ONNX và NCNN, dung lượng tăng lên đáng kể, đạt ngưỡng xấp xỉ 7 MB (ONNX: 7.01 MB, NCNN: 7.12 MB). Sự gia tăng này là hệ quả tất yếu của quá trình cấu trúc lại đồ thị tính toán, trong đó các công cụ biên dịch phải đóng gói thêm các cấu trúc tĩnh và thông tin đồ họa độc lập để đảm bảo mô hình có thể chạy tách rời khỏi môi trường Python ban đầu.

- Về tốc độ suy luận (Inference & FPS): Đây là khía cạnh phản ánh rõ nhất thành quả của kỹ thuật tối ưu hóa. Mô hình PyTorch tiêu tốn 114.104 ms để giải quyết một khung hình, chỉ đạt tốc độ 8.76 FPS. Khi sử dụng ONNX, thời gian suy luận giảm mạnh xuống còn 85.555 ms (tương đương 11.69 FPS). Đặc biệt, cấu trúc NCNN FP16 mang lại kết quả tốt nhất với 84.800 ms/ảnh và đạt 11.79 FPS. Việc lượng tử hóa trọng số xuống 16-bit đã giúp hệ thống tăng tốc độ xử lý khung hình lên gần 35% so với nguyên bản.

- Về độ chính xác (mAP50 và mAP50-95):

- + Tại ngưỡng đo mAP50, mô hình PyTorch dẫn đầu với chỉ số 80.226. Việc ép kiểu sang ONNX và NCNN khiến độ chính xác giảm vô cùng nhỏ (xuống mức 79.883 và 79.858). Mức suy giảm chưa tới 0.4% này là hoàn toàn nằm trong biên độ an toàn và cho phép đối với các bài toán triển khai nhúng.

- + Đáng chú ý, ở thang đo khắt khe hơn là mAP50-95 (yêu cầu độ trùng khớp hộp bao chuẩn xác hơn), các định dạng chuyển đổi ONNX (45.090) và NCNN (45.112) lại cho kết quả cao hơn bản gốc (43.255). Điều này chứng minh quá trình gộp lớp (layer fusion) và tối ưu hóa tính toán của trình biên dịch đã giúp mô hình loại bỏ các đặc trưng nhiễu, từ đó tổng quát hóa tốt hơn trên tập kiểm thử.

Dữ liệu thực nghiệm khẳng định việc chuyển đổi thuật toán từ môi trường nghiên cứu thuần túy (PyTorch) sang chuẩn tối ưu cho phần cứng biên (NCNN FP16) là một bước đi kỹ thuật hoàn toàn chính xác. Bằng việc hy sinh

một lượng rất nhỏ không gian lưu trữ, hệ thống đã tối đa hóa được tốc độ phân tích thời gian thực trong khi vẫn duy trì sự ổn định, thậm chí cải thiện độ chính xác ở các ngưỡng đo khắc khe. Đây là cơ sở thực tiễn để đảm bảo thuật toán có thể vận hành ổn định trên các vi mạch có quỹ tài nguyên hạn hẹp.

CHƯƠNG 7: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1. Tổng kết những kết quả đạt được

Đề tài đã hoàn thành mục tiêu nghiên cứu và triển khai một hệ thống cảnh báo cháy rừng tự động dựa trên thị giác máy tính và công nghệ vi máy tính nhúng. Về mặt tối ưu hóa thuật toán, nghiên cứu đã cấy ghép thành công kiến trúc mạng lai YOLOv7 kết hợp cùng xương sống MobileNetV3 và cơ chế chú ý tọa độ. Sự cải tiến này đã tạo ra một mô hình siêu nhẹ chỉ với 372 lớp xử lý và 332.253 tham số, giới hạn khối lượng tính toán ở mức 1.4 GFLOPs.

Mặc dù cấu trúc được thu gọn tối đa, thuật toán vẫn đảm bảo độ chính xác cao trên tập dữ liệu ảnh bị tước bỏ màu sắc, đạt mAP@50 ở mức 75.80%. Trong đó, khả năng định vị chính xác con người đạt 78.70%, theo sau là lớp lửa với 76.58% và lớp khói là 72.13%. Bên cạnh đó, đề tài cũng đã xây dựng hoàn chỉnh quy trình lượng tử hóa trọng số từ định dạng 32-bit xuống 16-bit và biên dịch sang cấu trúc NCNN. Nhờ vậy, tốc độ suy luận trên vi xử lý trung tâm của thiết bị biên đã vượt lên ngưỡng 18.00 FPS, tương đương 55.55 mili-giây cho mỗi khung hình.

Cuối cùng, một hạ tầng phần mềm khép kín đã được triển khai thành công thông qua môi trường ảo hóa trên Linux, kết hợp với giao thức truyền tải bảo mật và cấu trúc mã nguồn giải phóng bộ nhớ tự động để khoanh vùng nhận diện trực tiếp trên thiết bị thực tế.

7.2. Hạn chế của đề tài

Mặc dù hệ thống đã đạt được những kết quả khả quan, đề tài vẫn còn tồn tại một số rào cản kỹ thuật nhất định. Việc sử dụng thuần túy ảnh trắng đen tuy giúp tiết kiệm dung lượng luân chuyển nhưng lại vô tình làm mất đi các sắc màu quan trọng. Sự thiếu hụt này khiến bộ lọc tích chập đôi khi gặp khó khăn khi phân biệt dải khói mỏng với sương mù, hoặc ngọn lửa với các nguồn sáng nhân tạo có cường độ tương đương, dẫn đến chỉ số độ phủ tổng thể mới chỉ đạt 64.75%.

Hơn nữa, việc thuật toán phụ thuộc hoàn toàn vào quá trình tính toán trên CPU của bo mạch nhúng sẽ tạo ra giới hạn về năng lực thực thi. Hệ thống khó có thể duy trì tốc độ khung hình ổn định nếu thiết bị bay thay đổi vận tốc đột ngột hoặc khi cần nâng cấp độ phân giải của luồng video đầu vào. Một rủi ro khác nằm ở khâu hậu xử lý, nơi các vật thể có kích thước quá nhỏ như đốm lửa mới bùng phát hay người di chuyển ở khoảng cách cực xa rất dễ bị thuật toán triệt tiêu cực đại cực bộ đánh giá là nhiễu và lọc bỏ hoàn toàn khỏi kết quả dự đoán.

7.3. Hướng phát triển trong tương lai

Để hoàn thiện thuật toán và nâng cấp hệ thống trở thành một giải pháp phòng chống cháy rừng đáp ứng đầy đủ tiêu chuẩn công nghiệp, các hướng phát triển tiếp theo được đề xuất bao gồm:

- Tích hợp kiến trúc dữ liệu đa phương thức: Nâng cấp hệ thống tiếp nhận song song dữ liệu quang học và dữ liệu quang phổ hồng ngoại nhiệt (RGBT). Đặc tính phát xạ nhiệt sẽ giúp mô hình nhìn xuyên qua các lớp sương mù hoặc tán lá rừng rậm rạp, phát hiện chính xác các nguồn nhiệt ẩn bất chấp điều kiện ánh sáng hay ban đêm, từ đó cải thiện triệt để chỉ số Recall.

- Mở rộng hạ tầng viễn thông nhúng: Hoàn thiện kiến trúc IoT của thiết bị bằng cách tích hợp mô-đun truyền tải sóng vô tuyến tầm xa, áp dụng các nền tảng mạng như LoRa vào bo mạch vi điều khiển. Khi sự cố xảy ra, tọa độ vị trí cùng hình ảnh Bounding Box sẽ ngay lập tức được hệ thống mã hóa và phát tín hiệu báo động trực tiếp về trạm điều khiển mặt đất mà không cần phụ thuộc vào kết nối Internet di động.

TÀI LIỆU THAM KHẢO

- [1] Y. Zhang, X. Rui, và W. Song, "A UAV-Based Multi-Scenario RGB-Thermal Dataset and Fusion Model for Enhanced Forest Fire Detection," *Article*, [2025].
- [2] S. Ennaama và H. Silkan, "Enhanced Real-Time Object Detection using YOLOv7 and MobileNetv3," *Article*, [2024].
- [3] C.-Y. Wang, A. Bochkovskiy, và H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," *arXiv preprint arXiv:2207.02696*, 2022.
- [4] Q. Hou, D. Zhou, và J. Feng, "Coordinate Attention for Efficient Mobile Network Design," trong *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, tr. 13713-13722.
- [5] J. Jocher *et al.*, "Ultralytics YOLO," GitHub repository, 2026. [Trực tuyến]. Tại: <https://github.com/ultralytics/ultralytics>
- [6] Tencent, "NCNN: A high-performance neural network inference framework optimized for the mobile platform," GitHub repository, 2026. [Trực tuyến]. Tại: <https://github.com/Tencent/ncnn>
- [7] ONNX Runtime Developers, "ONNX Runtime: Cross-platform, high performance ML inferencing and training accelerator," GitHub repository, 2026. [Trực tuyến]. Tại: <https://github.com/microsoft/onnxruntime>